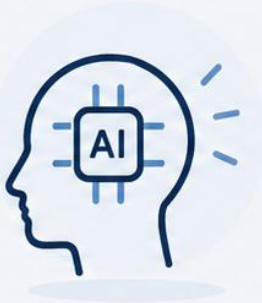
AI Agent Nedir?

AI Agent Nedir?

Tanım



AI Agent, belirli bir hedef doğrultusunda **karar verebilen**, gerektiğinde **araç kullanabilen** ve sonucu üretmek için **eylem gerçekleştirebilen** yapay zeka sistemidir.

Temel Özellikler

Hedef Odaklı
Belirli bir işi tamamlamaya çalışır.

Karar Verebilir
Duruma göre sonraki adımı belirler.

Eylem Alabilir
Bilgi toplar, işlem yapar ve çıktı üretir.

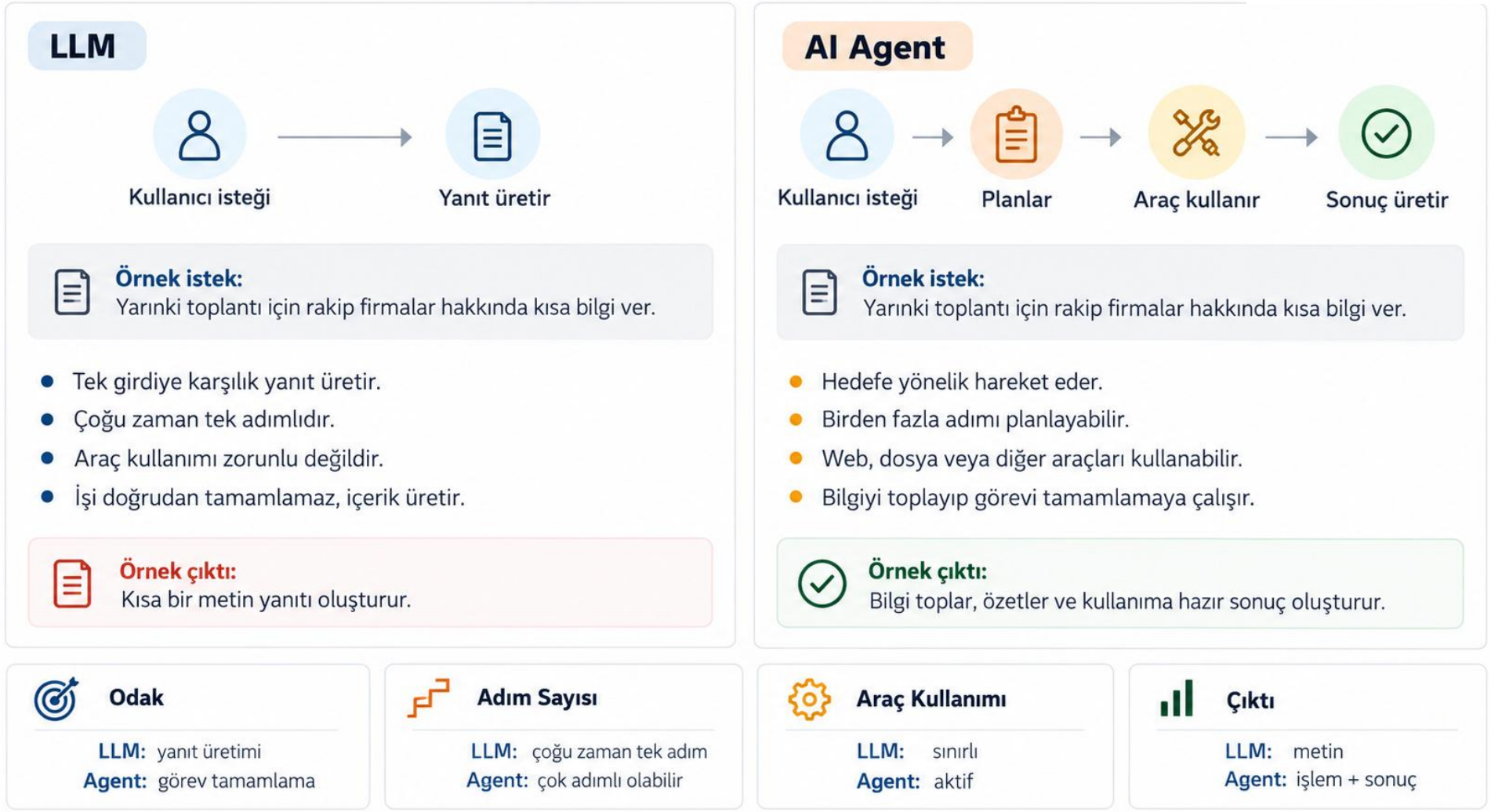
Kısa Gerçek Hayat Örnekleri

Toplantı Hazırlığı
Gündemi toplar, notları düzenler, kısa özet hazırlar.

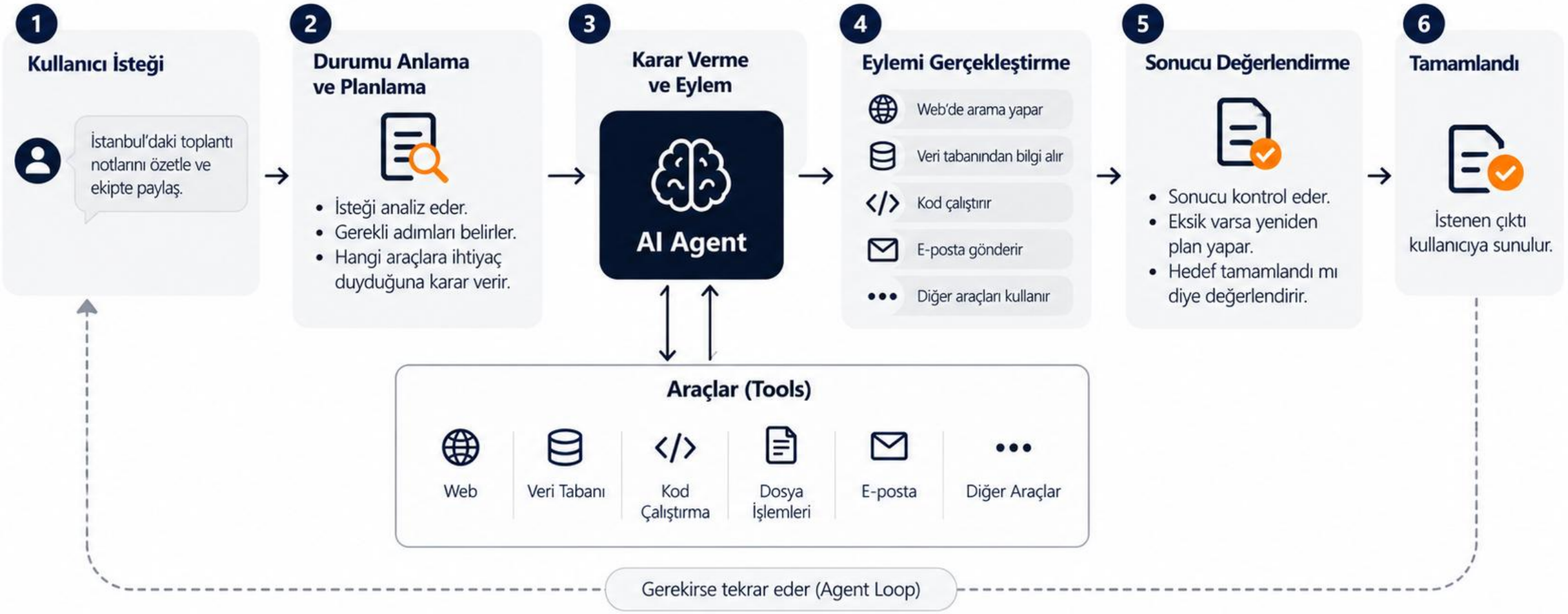
Müşteri Talebi
Talebi anlar, ilgili bilgiyi bulur, uygun yanıtı oluşturur.

Raporlama
Veriyi derler, önemli noktaları çıkarır, rapor oluşturur.

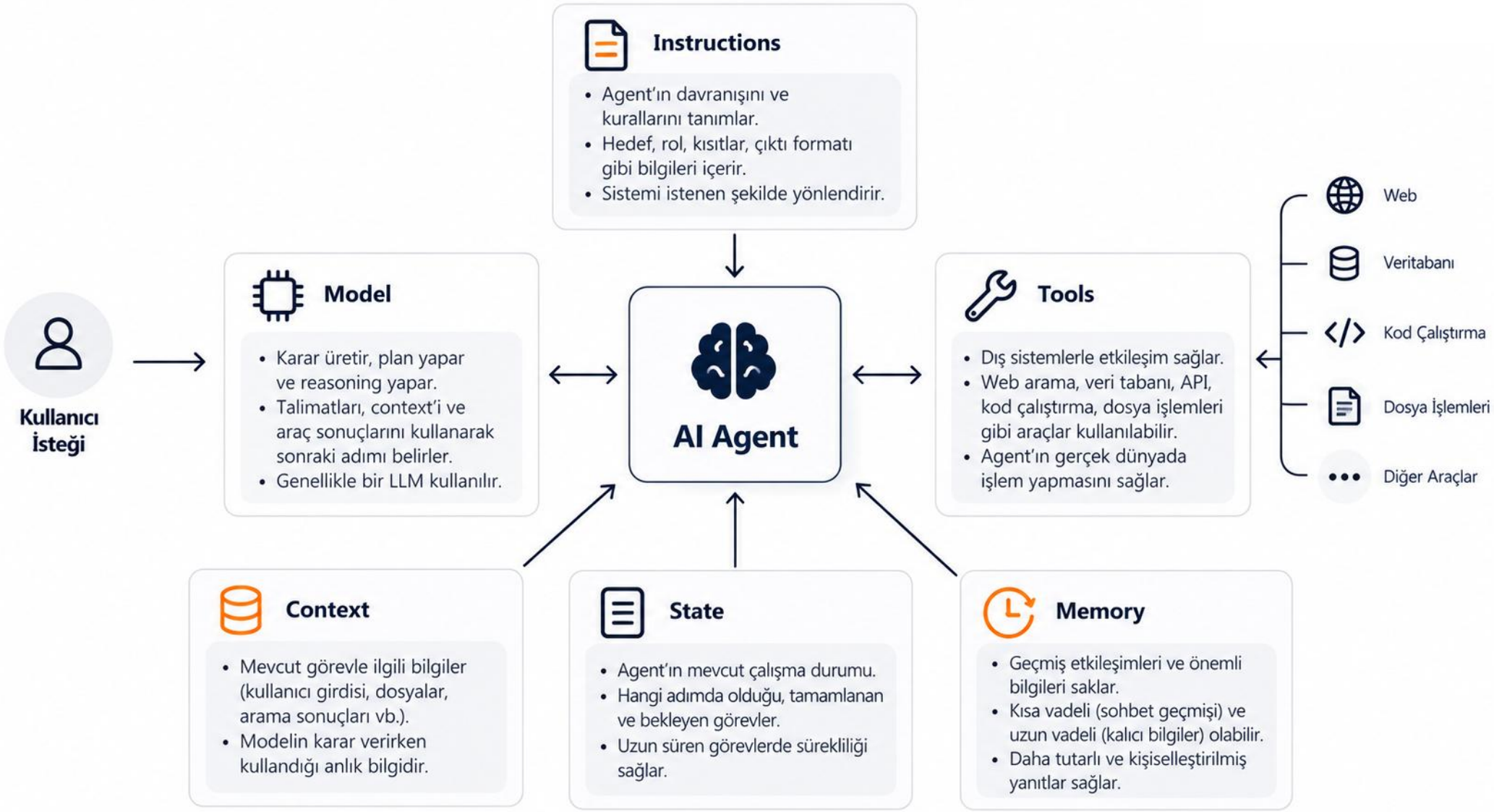
LLM ve AI Agent Arasındaki Fark



Agent Nasıl Çalışır?



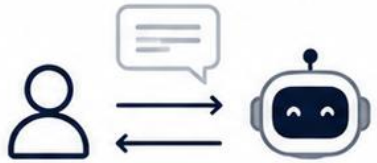
Agent'ın Temel Bileşenleri



Otonomi Seviyesi ve Agent Davranışı

Düşük Otonomi → Yüksek Otonomi

1 İnsan Yönlendirmeli



- Her adımı kullanıcı belirler.
- Model, verilen girdiye göre çıktı üretir.
- Bağımsız karar vermez.

Örnek
Kullanıcının verdiği metni özetle.

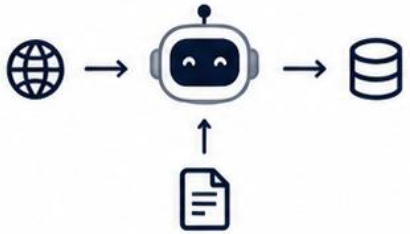
2 Kısmi Otonomi



- Hedefe ulaşmak için bazı adımları kendisi belirler.
- Belirli araçları kullanabilir.
- Kritik adımlarda kullanıcı onayı gerekebilir.

Örnek
Web'de arama yap, bulduğun bilgilerle kısa bir rapor hazırla ve bana göster.

3 Yüksek Otonomi



- Görevi tamamlamak için gerekli adımları kendisi planlar.
- Birden fazla aracı kullanabilir.
- Süreci izler ve gerektiğinde yeniden plan yapar.

Örnek
Pazar araştırması yap, verileri analiz et ve sonuçları rapor halinde sun.

4 Tam Otonomi



- Hedefe ulaşmak için gerekli tüm adımları kendi belirler.
- Birden fazla aracı ve veriyi kullanarak uçtan uca görevi tamamlar.
- Sadece sonuç raporlanır.

Örnek
Belirli bir konuda analiz yap, gerekli verileri topla, raporla ve aksiyon önerilerini sun.

AI Agent Kullanım Alanları

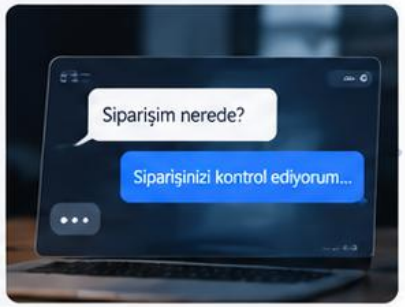


Araştırma ve Bilgi Toplama

- Web'de araştırma yapar
- Farklı kaynakları analiz eder
- Özet ve rapor oluşturur

Müşteri Hizmetleri

- Müşteri sorularını yanıtlar
- Talep ve şikayetleri yönetir
- Gerekli sistemlerde işlem yapar





Yazılım Geliştirme

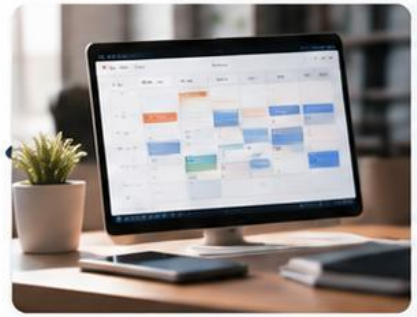
- Kod yazar ve düzenler
- Hata ayıklar ve test eder
- Proje üzerinde adımları takip eder

AI Agent

Veri Analizi

- Verileri toplar ve işleme alır
- Analizler yapar
- Grafik ve rapor üretir





Kişisel Asistanlar

- Takvim ve e-postayı yönetir
- Hatırlatıcılar oluşturur
- Günlük görevleri planlar

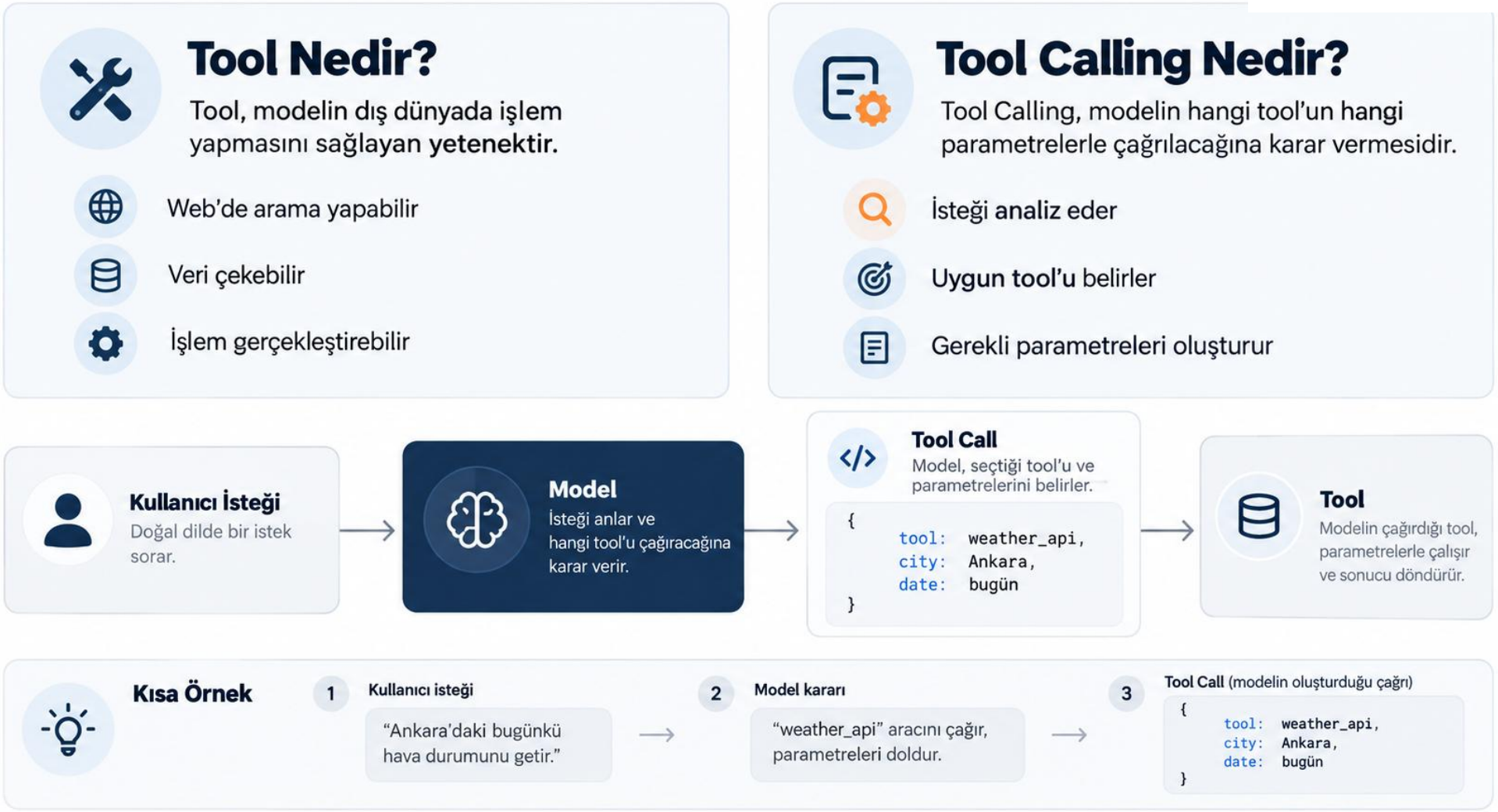
İş Süreçlerinin Otomasyonu

- Tekrarlayan görevleri otomatikleştirir
- Sistemler arasında işlem yapar
- İş akışlarını yönetir



Tool ve Tool Calling

Tool ve Tool Calling



Tool Tanımı: Description ve Schema

Description

Tool'un ne zaman ve hangi amaçla kullanılacağını açıklar.

- Hangi durumda kullanılacağını belirtir
- Modelin doğru tool'u anlamasını kolaylaştırır
- Kısa, net ve yönlendirici olmalıdır

ÖRNEK DESCRIPTION

```
"Bir şehir için güncel hava durumunu getirir.
Kullanıcı hava durumu sorduğunda kullanılır."
```

Schema

Tool'un hangi parametreleri, hangi veri tiplerinde beklediğini tanımlar.

- Parametre adlarını tanımlar
- Veri tiplerini belirtir
- Zorunlu ve opsiyonel alanları gösterir

ÖRNEK SCHEMA

```
{
    city: string,
    date: string,
    unit: string (opsiyonel)
}
```

Örnek Tool Tanımı

Description

```
"Belirli bir şehir için hava durumunu getirir.
Kullanıcı hava durumu sorduğunda kullanılır."
```

Schema

```
{
    city: "Ankara",
    date: "bugün"
}
```

Description kullanım amacını, **Schema** ise beklenen parametre yapısını tanımlar.

Birden Fazla Tool ve Tool Seçimi

Birden Fazla Tool

Tek bir model, farklı amaçlara hizmet eden birden fazla araca erişebilir.

Web Arama
Güncel bilgi ve dış kaynak araştırması

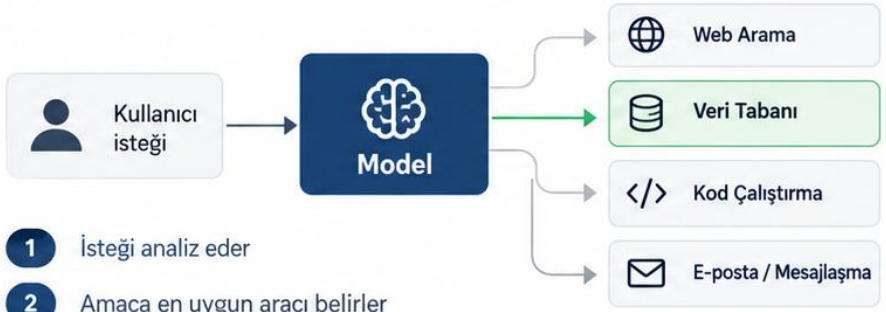
Veri Tabanı
Kayıt, tablo ve kurumsal veriye erişim

Kod Çalıştırma
Hesaplama, analiz ve veri işleme

E-posta / Mesajlaşma
Bildirim, gönderim ve iletişim işlemleri

Tool Seçimi

Tool Selection, mevcut görev için hangi aracın kullanılacağına karar verilmesidir.



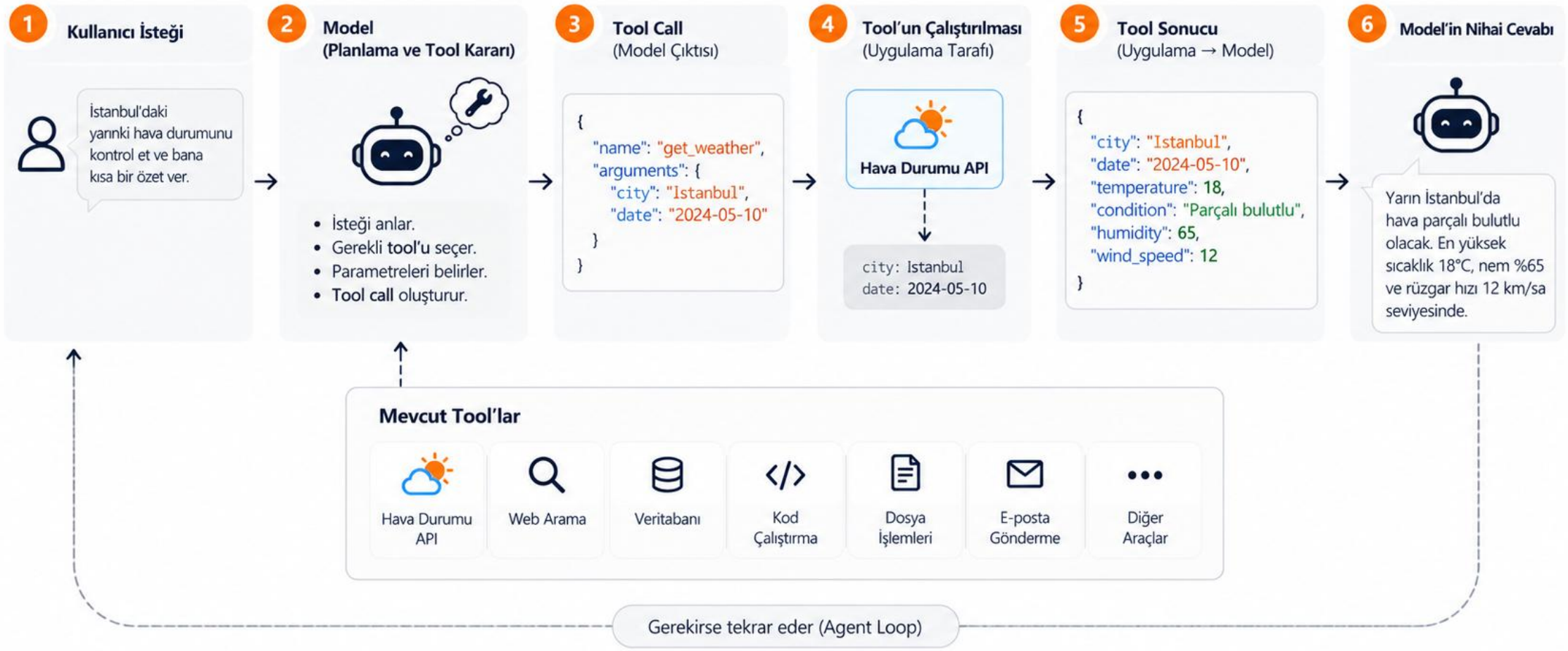
1. İsteği analiz eder
2. Amaca en uygun aracı belirler
3. Gerekirse birden fazla tool kullanabilir

Kısa Örnekler

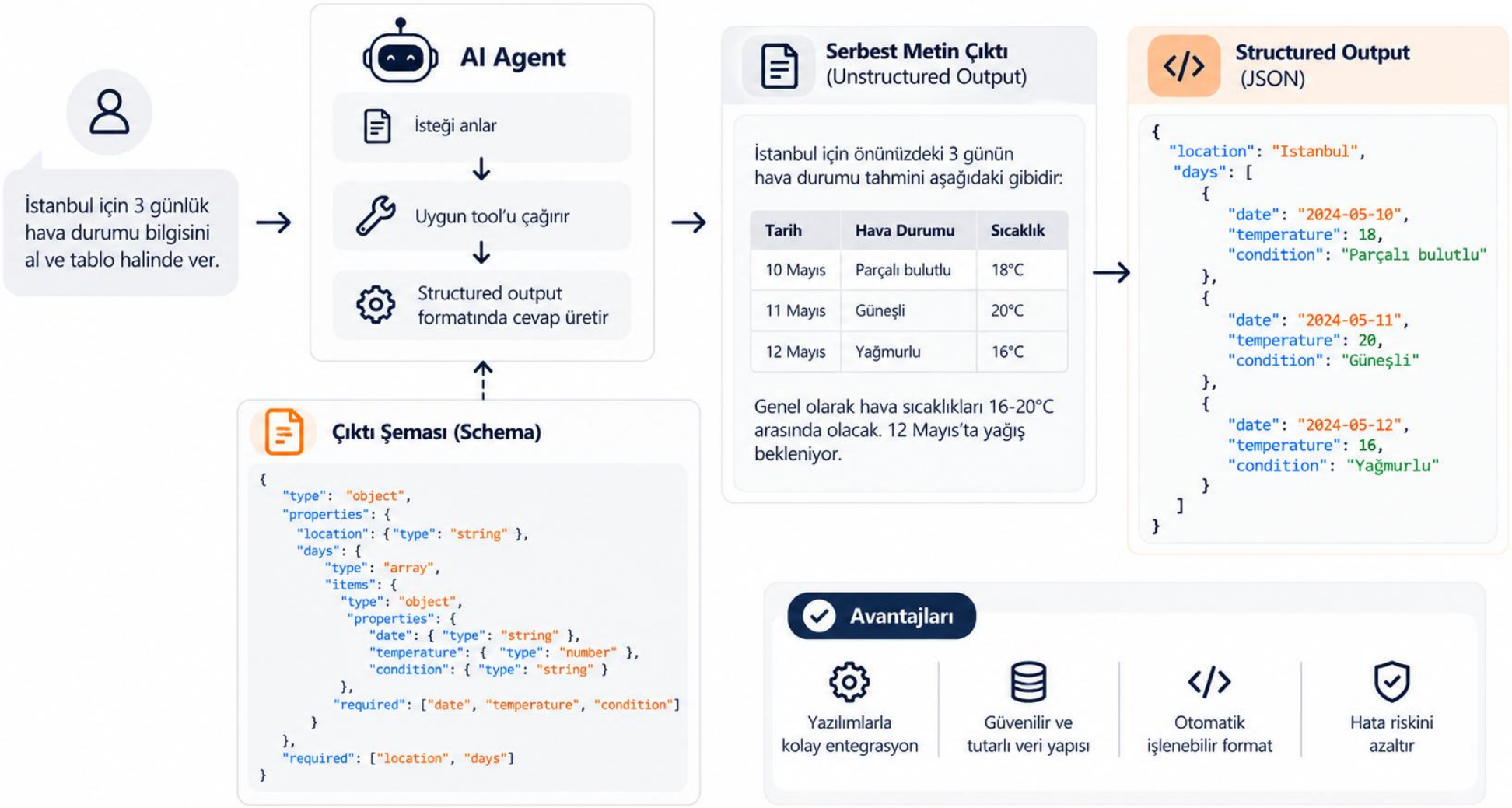
Kullanıcı isteği	Seçilen tool
Bugünkü dolar kurunu getir.	Web Arama
Son 3 ayın satış verisini bul.	Veri Tabanı
Bu verilerin ortalamasını hesapla.	Kod Çalıştırma

Her tool farklı bir amaç için vardır; model, göreve göre en uygun aracı seçer.

Tool Call Akışı



Structured Output



Workflow ve Agent Arasındaki Fark

Workflow ve Agent Arasındaki Fark

Workflow

Önceden belirlenmiş işlem adımlarını sırasıyla takip eden sistemdir.



Öne Çıkan Özellikler

- İşlem adımları önceden tanımlıdır.
- Her zaman aynı akış takip edilir.
- Karar noktaları sınırlıdır veya yoktur.
- Daha öngörülebilir ve kontrol edilebilir.
- Tekrarlanabilir, stabil süreçler için uygundur.

AI Agent

Görevin nasıl tamamlanacağına çalışma sırasında karar verebilen, esnek ve dinamik bir sistemdir.



Öne Çıkan Özellikler

- Adımlar çalışma sırasında belirlenir.
- Duruma göre farklı araçlar ve yöntemler kullanabilir.
- Sonuçları değerlendirip yeni kararlar verebilir.
- Esnek ve dinamik çalışır.
- Açık uçlu ve karmaşık görevler için uygundur.

Temel Workflow Desenleri

1 Prompt Chaining

Birden fazla LLM adımının ardışık olarak çalıştırılmasıdır.



Nasıl Çalışır?

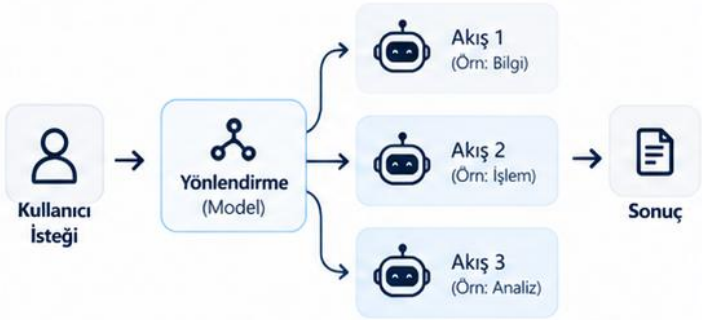
1. İlk LLM, görevi anlar ve bir ara çıktı üretir.
2. İkinci LLM, bu çıktıyı kullanarak bir sonraki adımı gerçekleştirir.
3. Son LLM, nihai sonucu oluşturur.

Ne Zaman Tercih Edilmeli?

- Görevin adımlara bölünebildiği durumlarda.
- Her adımın farklı bir uzmanlık gerektirdiği senaryolarda.
- Daha kontrollü ve öngörülebilir bir süreç istendiğinde.

2 Routing

Girdiyi analiz ederek uygun akışa yönlendiren bir desendir.



Nasıl Çalışır?

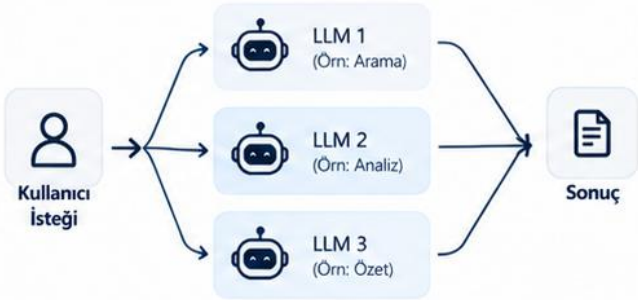
1. Model, kullanıcının isteğini analiz eder.
2. En uygun akışı belirler.
3. İsteği ilgili akışa yönlendirir ve sonucu döndürür.

Ne Zaman Tercih Edilmeli?

- Farklı konular veya niyetler için farklı işlemler gerektiğinde.
- Sınıflandırma veya yönlendirme ihtiyacı olduğunda.
- Doğru araca/uzmana hızlı ulaşmak istendiğinde.

3 Parallelization

Bağımsız işlemlerin eş zamanlı olarak çalıştırılmasıdır.



Nasıl Çalışır?

1. Görev, bağımsız alt görevlere bölünür.
2. Her alt görev eş zamanlı olarak çalıştırılır.
3. Sonuçlar birleştirilerek nihai çıktı oluşturulur.

Ne Zaman Tercih Edilmeli?

- Biribirinden bağımsız işlemler olduğunda.
- Hız ve verimlilik gerektiğinde.
- Paralel çalıştırmanın mümkün olduğu senaryolarda.

Tool Calling'den Agent'a Geçiş

Tool Calling

Tek bir işlemi gerçekleştirmek için modelin bir tool'u çağırmasıdır.



Özellikleri

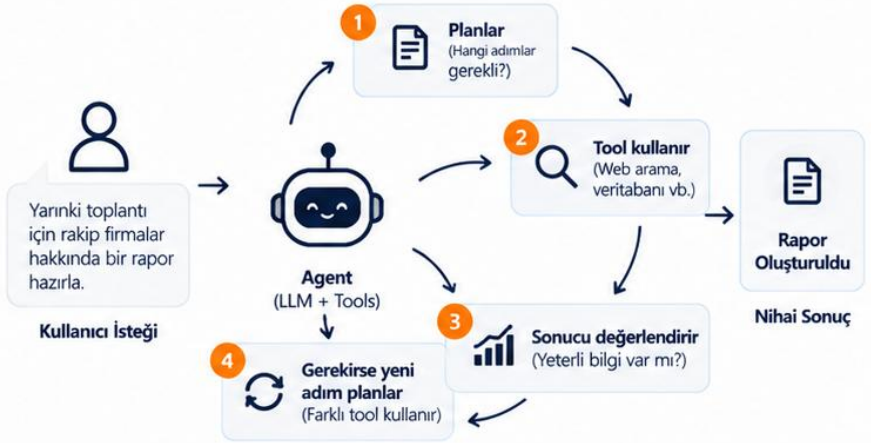
- Genellikle tek bir işlem için kullanılır.
- Model bir tool seçer ve parametreleri belirler.
- Tool çalıştırılır ve sonuç modele verilir.
- Ek adım planlaması veya dinamik karar verme yoktur.

Örnek Kullanım

- Hava durumu bilgisi alma
- Veri tabanından tek bir kayıt çekme
- Bir e-posta gönderme
- Basit bir hesaplama yapma

AI Agent

Görevi tamamlamak için birden fazla adımda karar verebilen, tool kullanabilen ve dinamik çalışan sistemdir.



Özellikleri

- Birden fazla adım planlayabilir.
- Sonuçlara göre yeni kararlar verebilir.
- Birden fazla tool kullanabilir.
- Görevi tamamlayana kadar döngüsel (agent loop) çalışır.
- Esnek ve dinamik bir yapıya sahiptir.

Örnek Kullanım

- Araştırma yapıp rapor oluşturma
- Farklı kaynaklardan veri toplayıp analiz etme
- Çok adımlı iş süreçlerini tamamlama
- Karar gerektiren ve duruma göre yaklaşım değiştiren görevler

Çok Adımlı Görevler ve Agent Loop



Stop Condition ve Sınırlar

Stop Condition

Stop Condition, agent loop'un tamamlandığını belirleyen koşuldur.



Durur
(koşullar sağlanırsa)

Devam eder
(koşullar sağlanmazsa)

Durma koşulu örnekleri

- Hedef tamamlandı
- Yeterli bilgi toplandı
- Kullanıcıya verilecek yanıt hazır

Koşul sağlanmazsa loop devam eder.

Execution Limits / Sınırlar

Execution Limits, agent'ın kontrolsüz veya gereksiz şekilde çalışmasını engelleyen sınırlardır.

Maksimum Adım Sayısı
En fazla kaç döngü çalışacağı belirlenir.

Süre Sınırı
Belirli bir süreden sonra işlem durdurulur.

Tool / İşlem Sayısı
Çağrılabilecek tool veya işlem sayısı sınırlandırılır.

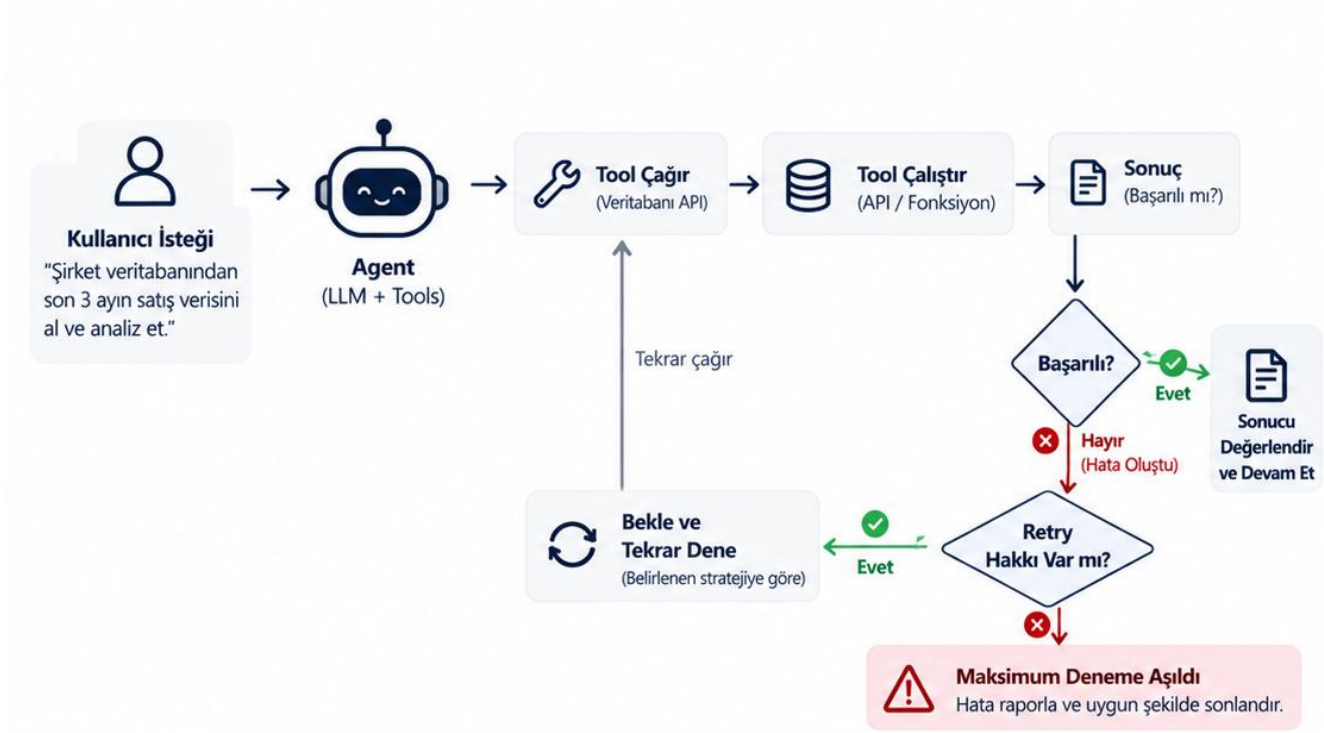
Maliyet / Kaynak Sınırı
Token, API veya işlem maliyeti kontrol altında tutulur.

Neden Gerekli?

- Kontrol sağlar
- Sonsuz döngüyü önler
- Maliyet ve zamanı sınırlar
- Daha güvenli çalışma sağlar

Stop condition agent'ın ne zaman duracağını, sınırlar ise ne kadar çalışabileceğini belirler.

Hata Yönetimi ve Retry



Olası Hata Türleri

Ağ / Bağlantı Hatası	API'ye ulaşılamaması veya ağ kesintisi.	**Örnek** ConnectionError Timeout
API Hatası	Sunucu tarafı hata veya yanıt alınamaması.	**Örnek** HTTP 500 Service Unavailable
Geçersiz Girdi Hatası	Yanlış veya eksik parametreler.	**Örnek** HTTP 400 Invalid parameters
Limit Aşımı	Rate limit, kota veya zaman aşımı.	**Örnek** HTTP 429 Rate limit exceeded
Beklenmeyen Hata	Tool veya model tarafında öngörülemeyen hatalar.	**Örnek** JSON parse error Unknown error

Retry Stratejileri

Sabit Aralık

Her denemede aynı süre beklenir.

```
1. deneme: 2 sn
2. deneme: 2 sn
3. deneme: 2 sn
```

Üssel Geri Bekleme (Exponential Backoff)

Her denemede bekleme süresi artar.

```
1. deneme: 1 sn
2. deneme: 2 sn
3. deneme: 4 sn
```

Rastgele Gecikme (Jitter)

Aşırı yüklenmeyi engellemek için rastgele süre eklenir.

```
2 sn + [0-1 sn]
4 sn + [0-2 sn]
8 sn + [0-4 sn]
```

Sınırlı Deneme Sayısı

Maksimum deneme sayısı belirlenir.

```
max_retries = 3
başarısızsa
işlem durdurulur
```

Dikkat Edilmesi Gerekenler

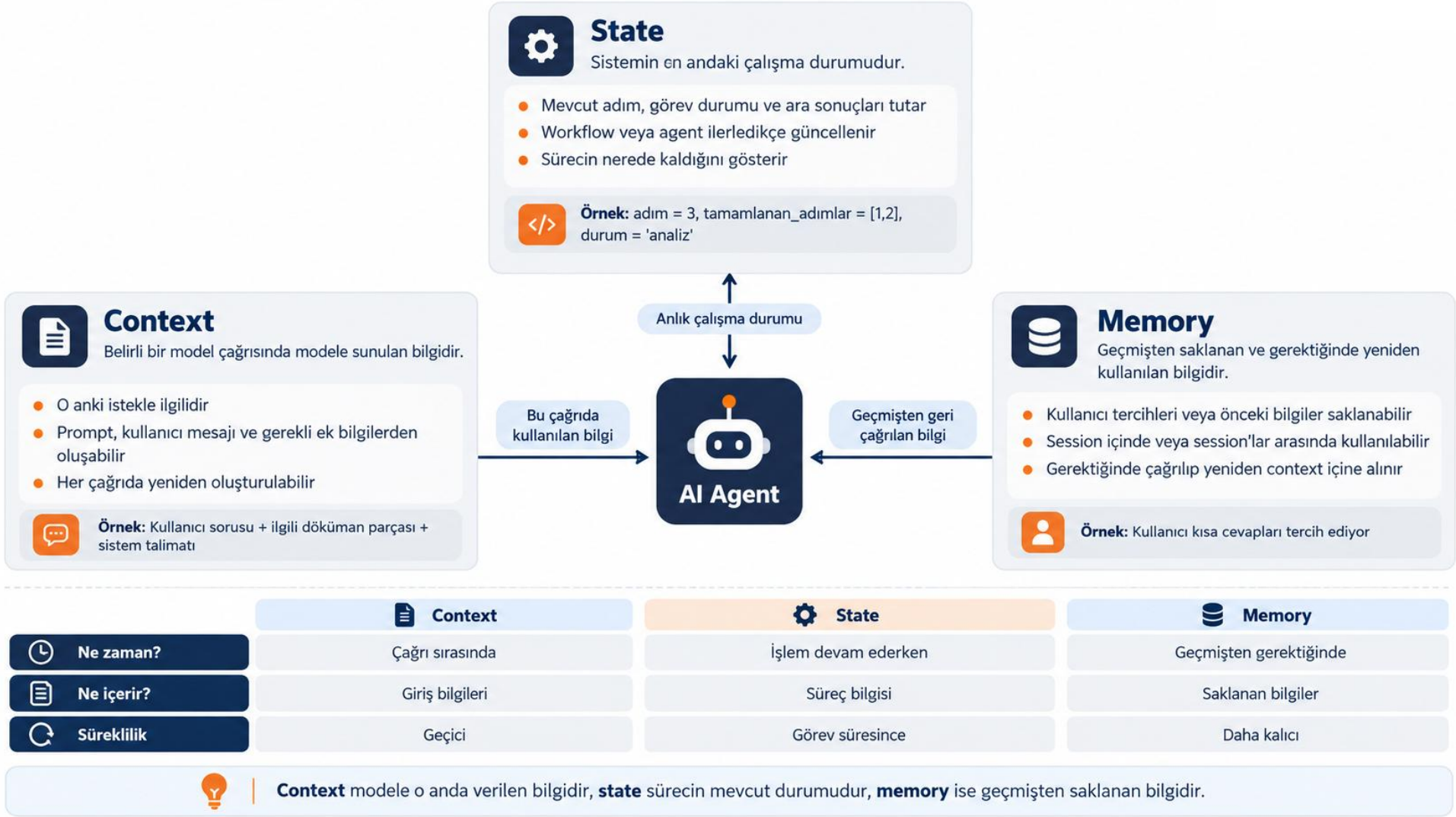
- Tüm hatalar için tekrar denemek uygun değildir (örn. geçersiz girdi).
- Maksimum deneme sayısı mutlaka sınırlandırılmalıdır.
- Bekleme süreleri, API limitlerine ve kullanım senaryosuna göre ayarlanmalıdır.
- Hata durumunda anlamlı bir mesaj üretilmeli ve gerekirse alternatif adımlar planlanmalıdır.

State Nedir?

State Nedir?


"Yarınki toplantı için rakip firmalar hakkında kısa bir rapor hazırla."
Kullanıcı İsteği
Agent
(LLM + Tools)
Web Araması
PDF İnceleme
Bilgileri Analiz Et
Rapor Oluştur
Rakip firma listesi
Firma A
Firma B
Firma C
Firma A özeti
Ürünler
Finansal durum
Son gelişmeler
Karşılaştırma
Güçlü yönler
Zayıf yönler
Fırsatlar
Nihai rapor
Özet
Karşılaştırma
Öneriler
State (Mevcut Durum)
Mevcut Adım
3 / 4 (Analiz)
Mevcut Görev
Rakip analizi raporu
Tamamlanan Adımlar
[1, 2]
Ara Sonuçlar
Firma listesi, özet bilgiler
Kullanılan Araçlar
[web_search, pdf_reader]
İşlem Durumu
Devam ediyor
Diğer Bilgiler
Hedef: 3 ana rakip, kısa rapor
Her adımda state güncellenir.
1
Önce rakip firmaları araştırıyorum.
mevcut_adim 1
gorev "Rakip analizi"
fonksiyon web_search
sonuclar ["Firma A","B","C"]
2
Firma A'nın son gelişmelerini inceliyorum.
mevcut_adim 2
gorev "Rakip analizi"
fonksiyon pdf_reader
sonuclar "Firma A özeti"
3
Firmaları karşılaştırıyorum.
mevcut_adim 3
gorev "Rakip analizi"
fonksiyon analysis
sonuclar "Karşılaştırma tablosu"
4
Raporu oluşturuyorum.
mevcut_adim 4
gorev "Rakip analizi"
fonksiyon report
sonuclar "Nihai rapor"

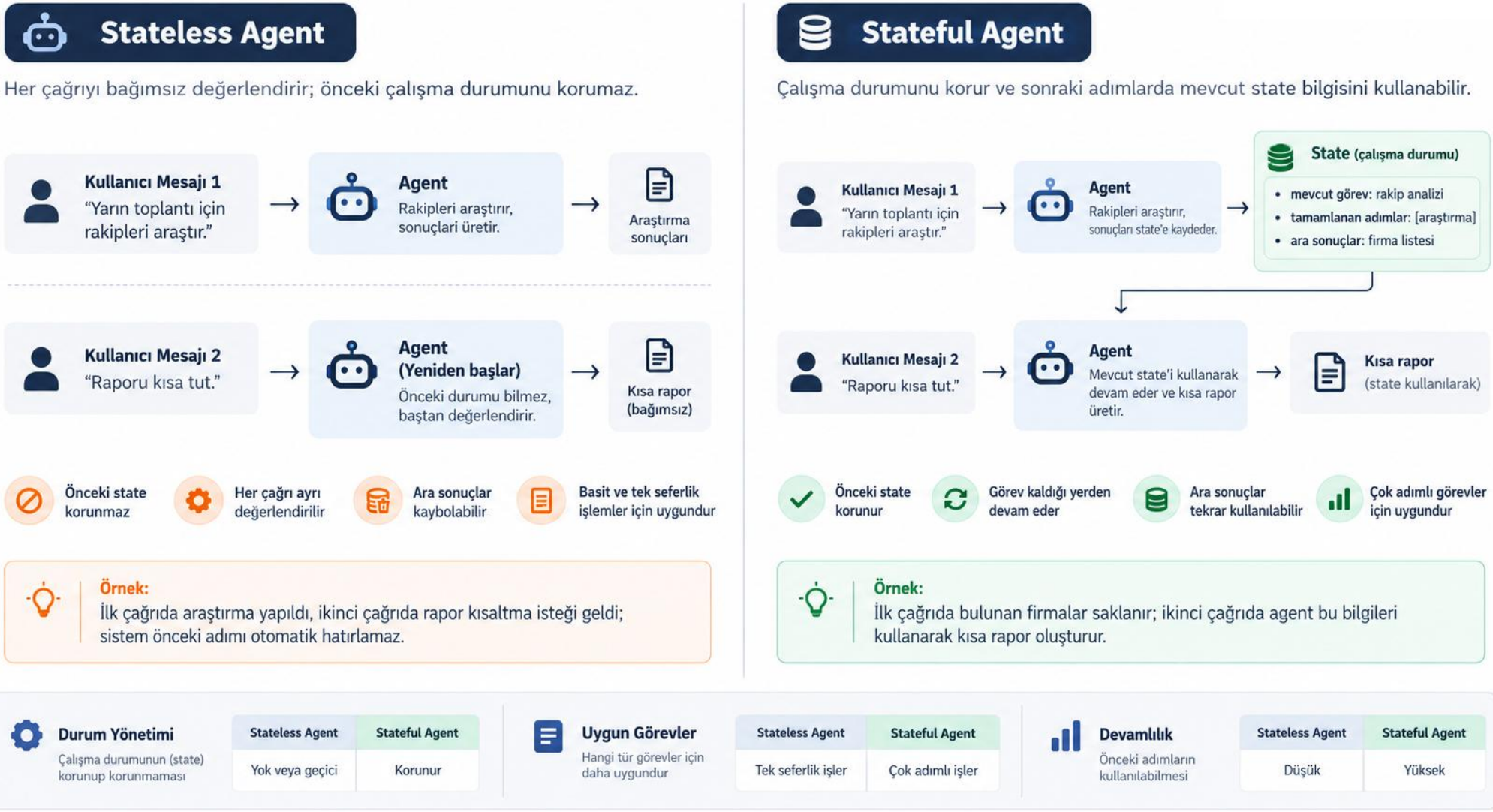
State, Context ve Memory Farkı



	Context	State	Memory
Ne zaman?	Çağrı sırasında	İşlem devam ederken	Geçmişten gerektiğinde
Ne içerir?	Giriş bilgileri	Süreç bilgisi	Saklanan bilgiler
Süreklilik	Geçici	Görev süresince	Daha kalıcı

Context modele o anda verilen bilgidir, **state** sürecin mevcut durumudur, **memory** ise geçmişten saklanan bilgidir.

Stateless ve Stateful Agent



	Stateless Agent	Stateful Agent
Durum Yönetimi — Çalışma durumunun (state) korunup korunmaması	Yok veya geçici	Korunur
Uygun Görevler — Hangi tür görevler için daha uygundur	Tek seferlik işler	Çok adımlı işler
Devamlılık — Önceki adımların kullanılabilmesi	Düşük	Yüksek

Short-Term Memory ve Conversation History



Sessions

Session, belirli bir kullanıcıya veya konuşmaya ait state ve geçmiş bilgilerin ilişkilendirildiği çalışma bağlamıdır. Her session, kendi conversation history, mevcut görev, ara sonuçlar ve diğer bilgilerle birlikte yönetilir.



Neden Session Kullanılır?

Kullanıcıları Ayırır
Farklı kullanıcıların konuşma geçmişi ve state bilgisi birbirinden bağımsız tutulur.

Tutarlı Deneyim Sağlar
Aynı session içinde önceki mesajlar ve ara sonuçlar kullanılabilir.

State ve Hafızayı İlişkilendirir
Belirli bir konuşmaya ait görev, state ve memory bilgileri aynı session altında saklanır.

Uzun Süreli Etkileşimleri Destekler
Kullanıcı farklı zamanlarda tekrar geldiğinde kaldığı yerden devam edebilir.

Long-Term Memory



Semantic Memory ve Vector Database

Semantic Memory

Kullanıcının ilgi alanları, tercihler, hedefleri, önceki etkileşimlerden çıkarılan anlamlı bilgilerin (facts, concepts) vektör temsilleriyle saklandığı uzun vadeli hafızadır.

Vector Database

Metin, belge veya çıkarılan bilgilerin embedding (vektör) olarak saklandığı ve anlamsal benzerlik aramasıyla hızlıca geri getirilebildiği veritabanıdır.

1. Bilgi (Metin)

- Yapay zeka ve görüntü işleme alanında çalışıyorum.
- Teknofest projelerine ilgi duyuyorum.
- Python ile çalışmayı seviyorum.

2. Embedding (Vektörleştirme)

[0.023, -0.18, 0.41, 0.56, ...]

- Metin, embedding modeli ile sayısal vektöre çevrilir.
- Benzer anlamdaki metinler vektör uzayında birbirine yakın konumlanır.

3. Vector Database

[0.023, -0.18, 0.41, ...]

[-0.12, 0.33, 0.27, ...]

[0.45, 0.11, -0.09, ...]

4. Anlamsal Arama

"Daha önce önerdiğin görüntü işleme projesini hatırlatır mısın?"

Query Embedding

[0.02, -0.17, 0.39, ...]

Benzerlik Araması (cosine similarity)

5. İlgili Bilgiler (Semantic Memory)

Geçen ay paylaştığım OpenCV projesini deneyebilirsin.	0.92
Yüz tanıma ile ilgili projeyi de önermiştim.	0.87
Python ile görüntü işleme kaynakları burada.	0.81

Neden Semantic Memory?

Kişiselleştirme
Kullanıcıyı daha iyi tanır, kişiselleştirilmiş yanıtlar üretiir.

Uzun Vadeli Hafıza
Geçmiş oturumlardan edinilen bilgiler saklanır ve tekrar kullanılır.

Anlamsal Arama
Anahtar kelime yerine anlama dayalı arama ile daha ilgili sonuçlar getirilir.

Daha Doğru Kararlar
Kulharıcının tercih ve hedeflerine göre daha uygun öneriler sunulur.

Saklanabilecek Bilgiler ve Kaynaklar

Kullanıcı Bilgileri
- Tercihler
- İlgi alanları
- Hedefler
- Geçmiş işler / projeler

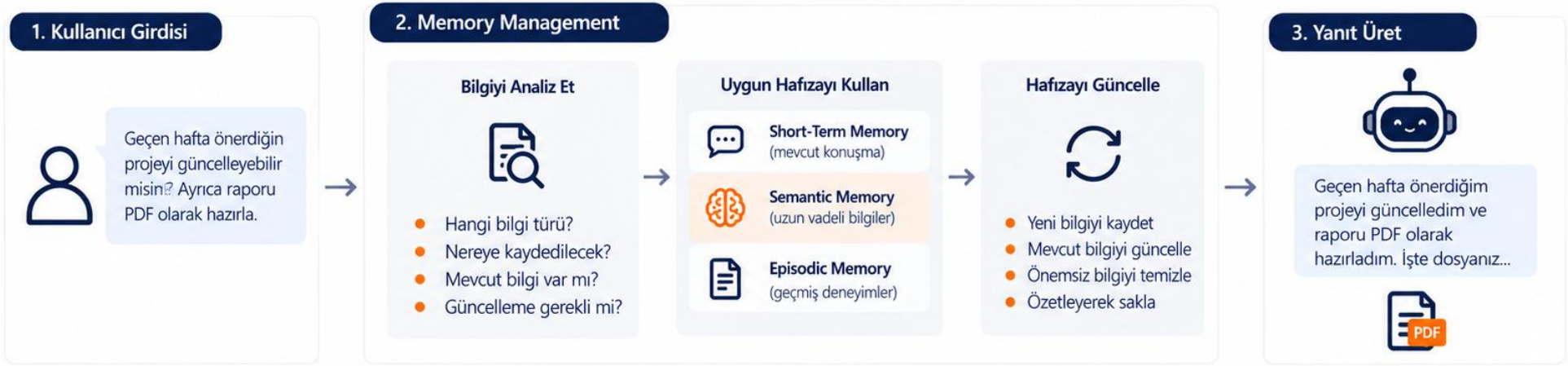
Belgeler
- PDF / DOC / TXT
- Notlar
- Raporlar
- Kurumsal dokümanlar

Konuşma Geçmişi
- Önemli bilgiler
- Çıkarılan özetler
- Kullanıcı soruları
- Verilen tavsiyeler

Harici Kaynaklar
- Web sayfaları
- Veritabanları
- API çıktıları
- Diğer sistemler

Memory Management

Memory Management, agent'in hangi bilgiyi ne zaman, nasıl saklayacağını, nasıl kullanacağını ve gerektiğinde nasıl güncelleyeceğini belirleyen süreçtir. Amacı, doğru bilgiyi doğru zamanda kullanarak daha tutarlı, verimli ve güvenilir yanıtlar üretmektir.



Temel Stratejiler

Bilgi Seçimi
Sadece önemli ve ilgili bilgileri sakla.

Özetleme
Uzun konuşmaları özetleyerek kaydet.

Etiketleme / Kategorizasyon
Bilgileri konu, tarih, kullanıcı gibi etiketlerle organize et.

Temizleme
Gereksiz veya güncelliğini yitirmiş bilgileri sil.

Hafıza Türlerine Göre Yönetim

Short-Term Memory
- Mevcut oturumdaki konuşma geçmişi
- Geneliikle otomatik yönetilir

Semantic Memory
- Kullanıcı tercihleri, hedefler, önemli bilgiler
- Özetlenerek vektör veritabanında saklanır

Episodic Memory
- Geçmiş etkileşimler, yapılan işler, deneyimler
- Zaman bilgisi ile birlikte saklanır

Örnek: Bilgi Güncelleme Akışı

Artık görsel analiz yerine metin analizi projelerine öncelik ver.

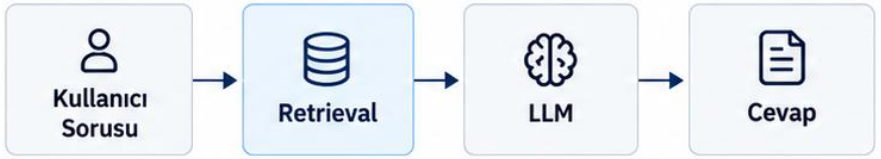
Agent
Memory Management

- Yeni tercihi kaydet
- Önceki tercihi güncelle
- Konu etiketle ("tercihler")
- Semantic Memory'ye ekle (vektör DB)

Agentic RAG ve Klasik RAG Farkı

Agentic RAG ve Klasik RAG Farkı

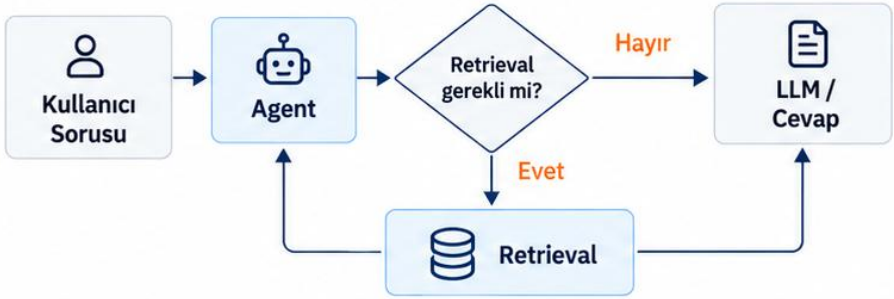
Klasik RAG



1. Retrieval akışı sabittir.
2. Bilgi getirme adımı genellikle her soruda çalışır.
3. Pipeline önceden tanımlıdır.
4. Karar verme esnekliği düşüktür.

Örnek: Kullanıcı soru sorar, sistem doğrudan vektör veritabanından bilgi getirir ve cevabı üretir.

Agentic RAG



1. Retrieval kararı dinamik olarak verilir.
2. Agent, ne zaman bilgi getireceğine karar verir.
3. Bilgi getirme süreci daha esnek yönetilir.
4. Gerekirse retrieval yeniden tetiklenebilir.

Örnek: Agent önce elindeki bilginin yeterli olup olmadığını değerlendirir; gerekirse bilgi getirir, sonra cevabı oluşturur.

Akış

Klasik RAG	Agentic RAG
Sabit	Dinamik

Retrieval Tetikleme

Klasik RAG	Agentic RAG
Genellikle otomatik	İhtiyaca göre

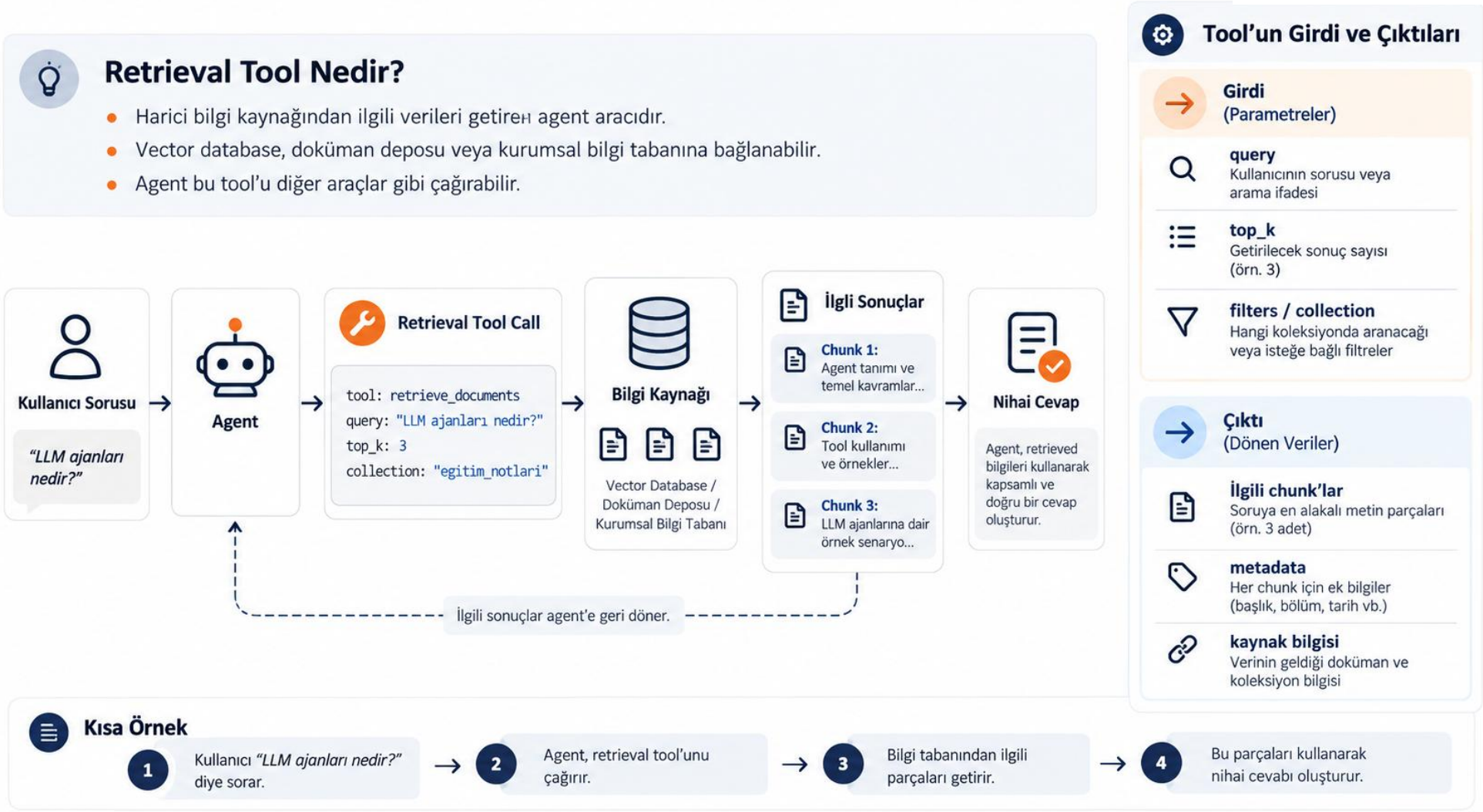
Esneklik

Klasik RAG	Agentic RAG
Düşük	Yüksek

Kontrol

Klasik RAG	Agentic RAG
Pipeline odaklı	Agent odaklı

Retrieval'ı Tool Olarak Kullanmak



Retrieval Kararı

Retrieval Kararı, agent'ın mevcut bilgilerinin soruyu cevaplamak için yeterli olup olmadığını değerlendirmesi ve gerektiğinde retrieval yapıp yapmayacağına karar vermesidir.

Kullanıcı Sorusu

"RAG mimarisinde reranking yöntemleri nasıl çalışır?"

Agent
(LLM + Tools)

Karar Süreci

1. **Soruyu Anla**
   Kullanıcının ne sorduğunu analiz et.
2. **Mevcut Bilgiyi Değerlendir**
   Modelin kendi bilgisi yeterli mi?
3. **Retrieval Gerekli mi?**
   Karar ver ve planla.

Retrieval Yapma (Gerek Yok)

"Reranking, ilk aşamada getirilen sonuçları ilgileine göre yeniden sıralama yöntemidir..."

Ne Zaman?

- Genel bilgi soruları
- Modelin zaten bildiği konular
- Güncel bilgi gerektirmeyen sorular
- Açıklama / tanım soruları

Retrieval Yap (Gerekli)

Sorguyu iyileştir → İlgili bilgileri retrieve et → Sonuçları değerlendir → Cevabı üret

Ne Zaman?

- Şirket içi / özel dokümanlar
- Alan spesifik veya teknik sorular
- Güncel ve değişen bilgiler
- Modelin bilmediği konular
- Kaynak gösterilmesi gereken yanıtlar

Retrieval Kararı İçin Değerlendirme Kriterleri

1. Soru Tipi
- Genel mi, teknik mi?
- Tanım mı, özel bilgi mi?
- Güncel bilgi gerekiyor mu?

2. Modelin Mevcut Bilgisi
- Model bu konuda yeterli bilgiye sahip mi?
- Hatalı cevap riski var mı?

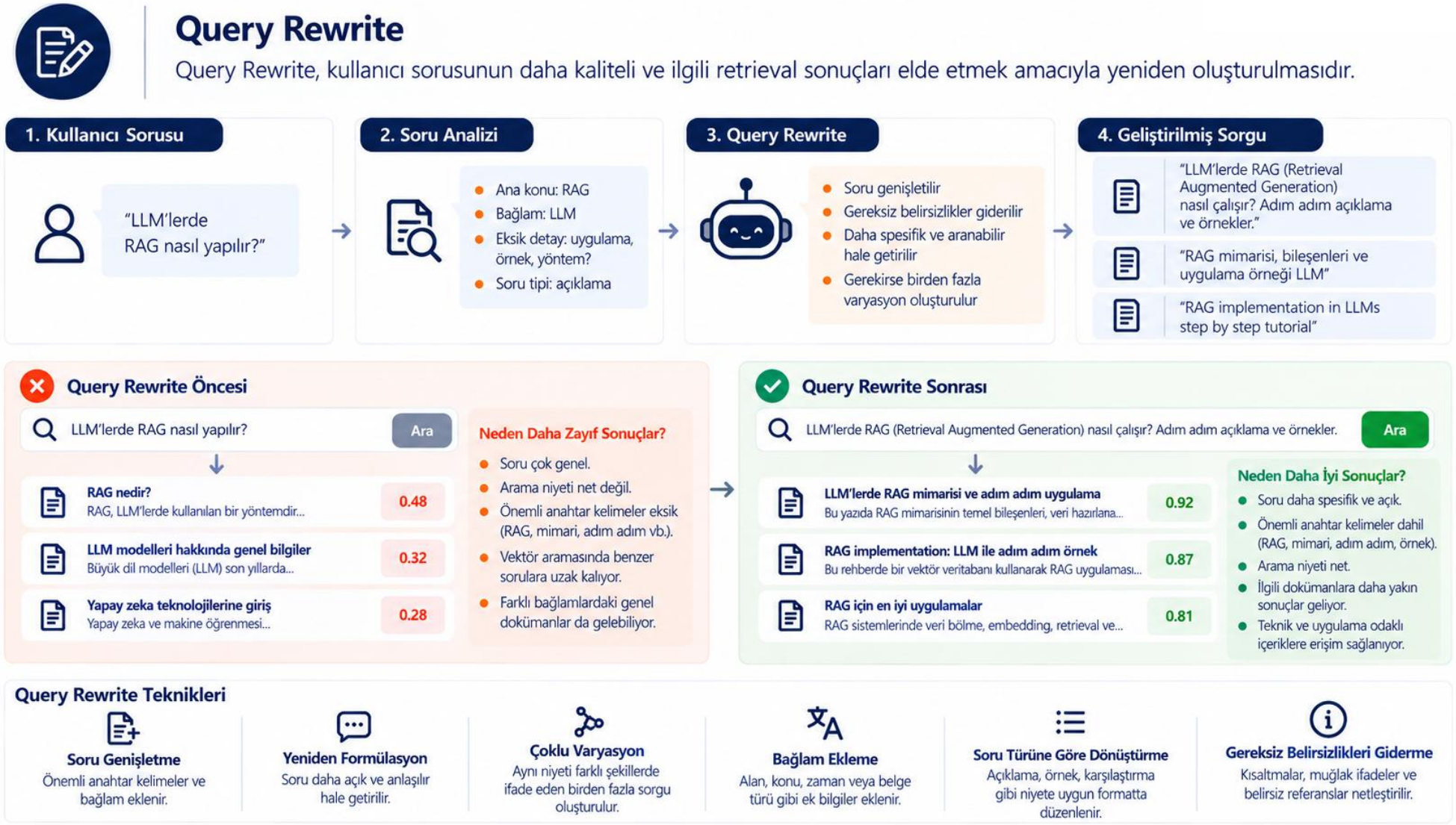
3. Güncellik İhtiyacı
- Bilgi zamanla değişiyor mu?
- Güncel veri gerekiyor mu?

4. Güvenilirlik
- Doğruluk kritik mi?
- Kaynak gösterilmesi gerekiyor mu?

Örnek Senaryolar

Kullanıcı Sorusu	Retrieval Kararı	Neden?
"RAG nedir?"	Gerek Yok	Genel bilgi, model biliyor.
"Firmamızın iade politikasını özetler misin?"	Gerekli	Şirket içi doküman, modele özel bilgi değil.
"2024'teki yeni düzenlemeler nelerdir?"	Gerekli	Güncel bilgi gerekiyor.
"Cosine similarity ne demek?"	Gerek Yok	Genel teknik bilgi.
"Bu PDF'de geçen ana başlıklar neler?"	Gerekli	Belgeye özel içerik, retrieval gerek.

Query Rewrite



Kaynak Seçimi

Kaynak seçimi, agent'ın mevcut görev için en uygun bilgi kaynağını (vector database, web, SQL veya farklı kaynaklar) değerlendirerek hangisini kullanacağına karar vermesidir.



Farklı Sorular, Farklı Kaynak Seçimleri

"Şirketimizin iade politikasını özetle."	"Yapay zeka alanındaki güncel gelişmeler nelerdir?"	"Son 6 ayın satış rakamlarını getir."	"Bu görseldeki tabloyu analiz et."
Vector Database Şirket dökümanları ve bilgi tabanı	**Web Arama** Güncel ve değişen bilgiler	**SQL Veritabanı** Yapısal ve sayısal veriler	**Diğer Kaynaklar** API'ler, dosyalar, üçüncü taraf servisler

Retrieval Sonucunu Değerlendirme



Multi-Step Retrieval



Context Engineering Nedir?

Context Engineering Nedir?

Context Engineering, modelin doğru karar verebilmesi için doğru bilgiyi, doğru zamanda, doğru miktarda ve uygun düzenle sunma disiplinidir.

Görev / Kullanıcı İsteği

"Bu ayki satış düşüşünü özetle."

Context Tasarımı

Doğru bilgi (alaka)
Görev için gerçekten gerekli bilgiler seçilir.

Doğru zaman (zamanlama)
Bilgi ihtiyaç duyulan anda modele verilir.

Doğru miktar (yeterlilik)
Eksik ya da aşırı değil, yeterli context sağlanır.

Doğru düzen (öncelik)
Önemli bilgiler açık ve öncelikli şekilde sunulur.

Model Kararı / Yanıt

Daha doğru, odaklı ve ilgili yanıt

Neden Önemli?

- Daha tutarlı kararlar
- Daha ilgili yanıtlar
- Gereksiz bilgi yükünü azaltma
- Tool, retrieval ve memory kullanımını daha etkili hale getirme
- Hata ve dikkat dağınıklığını azaltma

Kısa Örnek

Kullanıcı isteği "Bu ayki satış düşüşünü özetle."

İyi context tasarımı
- Son satış raporu
- İlgili tablo ve temel metrikler
- Kısa görev talimatı
- Gereksiz geçmiş mesajlar hariç

Sonuç Model daha odaklı ve doğru bir özet üretir.

Anahtar Kavramlar

Seçim
Hangi bilgi dahil edilecek?

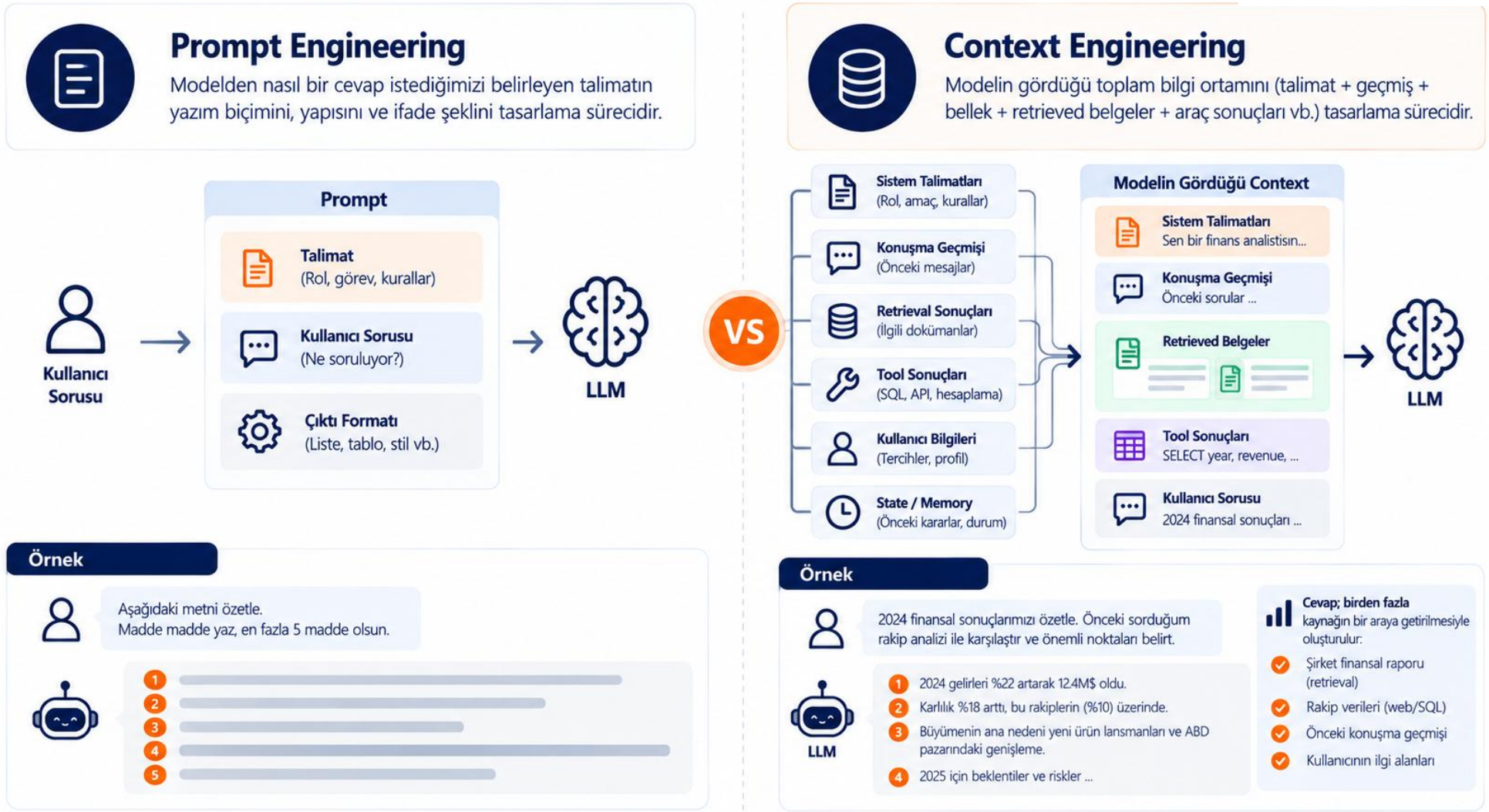
Filtreleme
Gereksiz bilgi dışarıda kalır.

Odak
Modelin dikkati ilgili bilgiye yönelir.

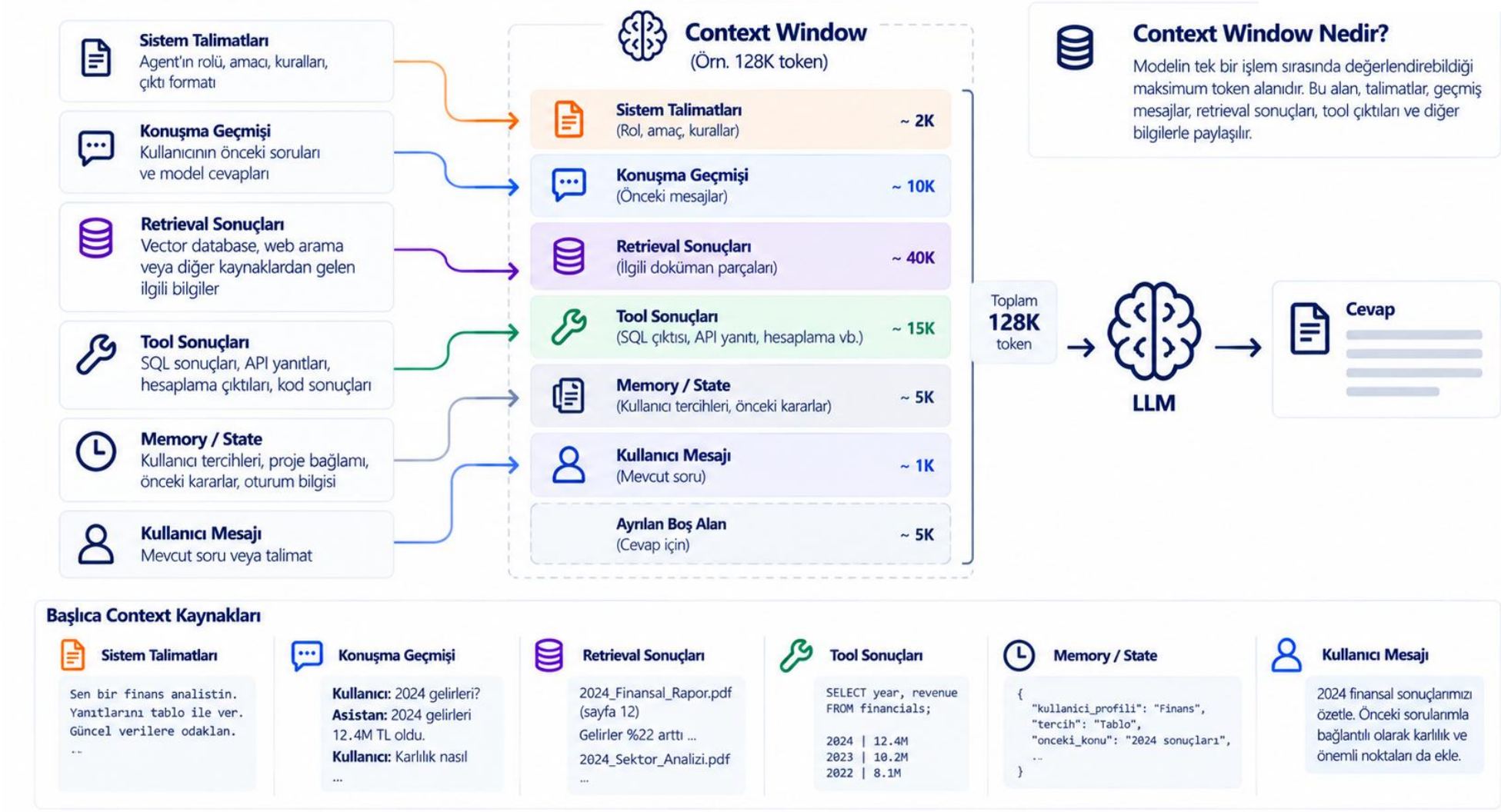
Netlik
Bilgi anlaşılır biçimde sunulur.

Karar Kalitesi
Daha iyi yanıt ve aksiyon üretimi

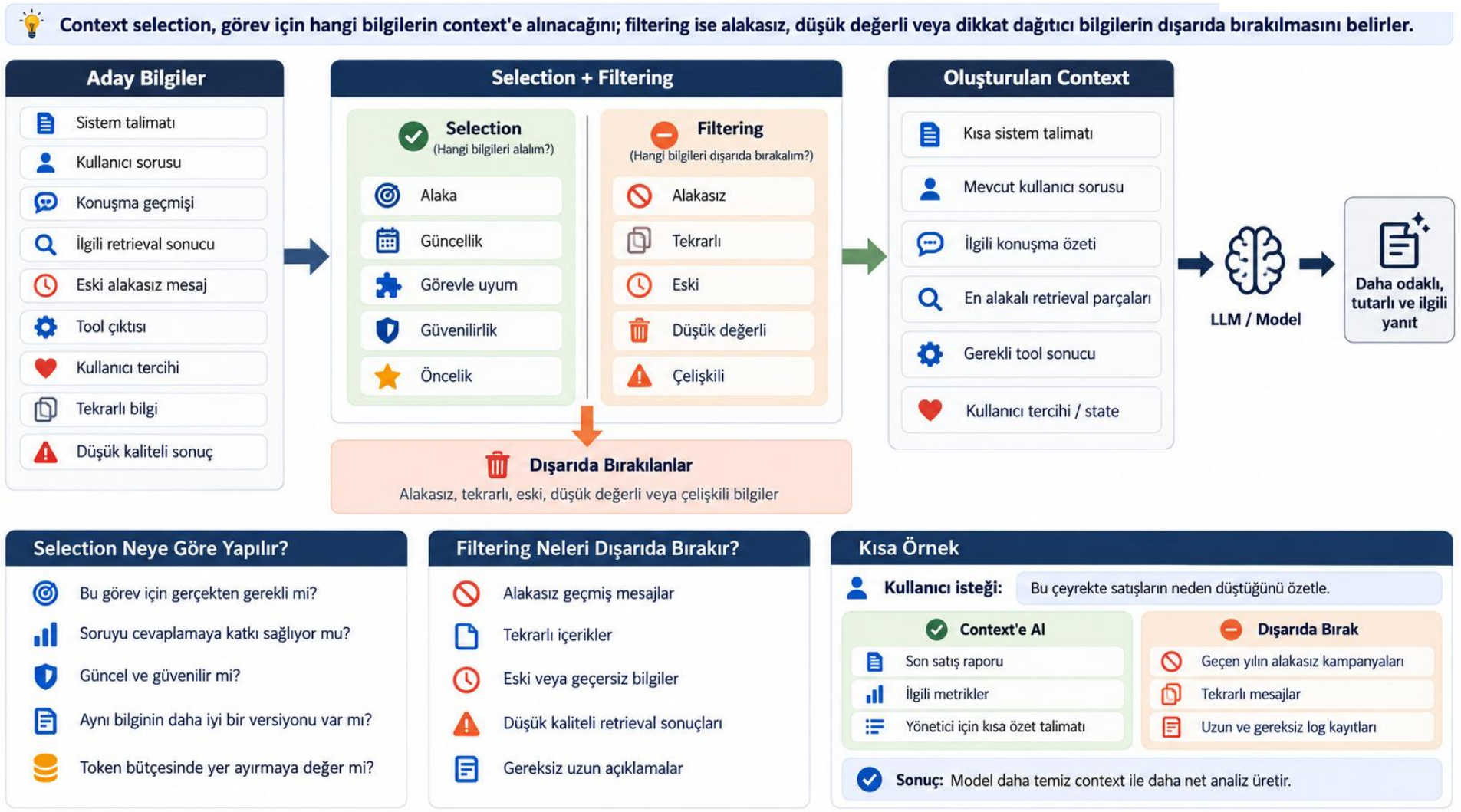
Prompt Engineering ve Context Engineering Farkı



Context Window ve Context Kaynakları



Context Selection ve Filtering



Context Trimming ve Compression

Context trimming, uzayan context içindeki eski, düşük öncelikli veya artık gerekli olmayan parçaların çıkarılmasıdır; compression ise önemli bilginin anlam korunarak daha kısa bir temsile dönüştürülmesidir.

Trimming (çıkarma)

Eski veya düşük öncelikli içerik context'ten tamamen çıkarılır.

Uzun Context (Öncesi)

- Sistem talimatı
- Mevcut kullanıcı sorusu
- Son retrieval sonucu
- Son tool çıktısı
- Eski konuşma mesajları
- Tekrarlı açıklamalar
- Eski log kayıtları

Trimmed Context (Sonrası)

- Kısa sistem talimatı
- Mevcut kullanıcı sorusu
- Son retrieval sonucu
- Gerekli tool çıktısı

Neler çıkarılabilir?

- Eski mesajlar
- Tekrarlı içerik
- Düşük öncelikli detaylar
- Artık gerekli olmayan ara çıktılar

Etkisi

Token kullanımını azaltır ve modeli güncel göreve odaklar.

Compression (sıkıştırma)

Uzun içerik atılmaz; daha kısa ama anlamı korunmuş bir temsile dönüştürülür.

Uzun içerik (Öncesi)

Satışlar Q2'de düştü. En büyük neden distribütör kanalındaki daralma ve iki büyük müşteride sipariş ertelemesiydi. Pazarlama maliyetleri arttı, kârlılık baskılandı. Ayrıca bazı bölgelerde stok sorunları yaşandı ve yeni ürün lansmanı planlandığı gibi ilerlemedi. Rekabetin artması da fiyatlar üzerinde baskı oluşturdu. Genel olarak, hem iç hem de dış faktörler bir araya gelerek Q2'deki düşüşte etkili oldu.

Compressed Context (Sıkıştırılmış özet)

- **Q2 satış düşüşü**
- **Neden:** distribütör daralması + sipariş ertelemesi
- **Kârlılık:** artan pazarlama maliyeti nedeniyle baskılandı

Ne korunur?

- Ana anlam
- Temel karar bilgisi
- Kritik sayılar / bulgular

Etkisi

Daha az token ile önemli bilgiyi korur.

Kısa Karşılaştırma

Özellik	Trimming (çıkarma)	Compression (sıkıştırma)
Amaç	Gereksiz içeriği kaldırmak	Uzun içeriği kısaltmak
İşlem	Silme / çıkarma	Özetleme / yeniden temsil
Ne olur?	Bazı içerik tamamen gider	İçerik kalır ama daha kısa hale gelir

Kısa Örnek

Kullanıcı isteği: "Bu çeyrekte satışların neden düştüğünü özetle."

Trimming (çıkarma): Geçen yılın alakasız kampanya konuşmaları çıkarılır.

Compression (sıkıştırma): Uzun satış toplantısı notları kısa özet haline getirilir.

Sonuç: Model daha kısa ve daha faydalı bir context ile çalışır.

Context Budgeting

Context budgeting, kullanılabilir context kapasitesinin instructions, history, state, memory, retrieval ve tool sonuçları arasında bilinçli biçimde dağıtılmasıdır.

Toplam Context Bütçesi



Bütçe Dağıtımı Nasıl Yapılır?

Görev türü
Analiz, özet, kodlama, çok adımlı görev gibi farklı türler farklı alan ihtiyacı doğurur.

Güncellik ihtiyacı
Daha güncel bilgi gerekiyorsa retrieval ve tool sonuçlarına daha fazla alan ayrılır.

Kaynak sayısı
Kullanılacak doküman, veri tablosu, rapor ve araç sayısı arttıkça ilgili bölümlere daha fazla bütçe gerekir.

Beklenen çıktı uzunluğu
Kısa yanıtlar daha az, detaylı raporlar daha fazla alan gerektirir.

Model sınırı ve maliyet
Modelin maksimum context sınırı ve token maliyeti, dağıtımı belirler.

Örnek Görev

"2024 satış düşüşünün nedenlerini, rakiplerle karşılaştırarak kısa bir yönetici özeti halinde hazırla."

Örnek Bütçe Planı

1. Retrieval için daha fazla alan ayrılır.
2. İlgili tablo ve rapor parçaları alınır.
3. Uzun geçmiş yerine kısa konuşma özeti kullanılır.
4. Cevap için yeterli alan bırakılır.
5. Gereksiz tool çıktıları eklenmez.

İyi Bütçeleme Ne Sağlar?

Daha ilgili context
En önemli bilgiler için yeterli alan ayrılır.

Daha az dikkat dağınıklığı
Gereksiz bilgiler yerine görevle ilgili içerik öne çıkar.

Daha tutarlı yanıt
Model kritik bilgileri kaçırmadan daha dengeli yanıt verir.

Token verimliliği
Aynı bütçe ile daha yüksek fayda sağlanır.

Daha iyi araç ve retrieval kullanımı
Araç sonuçları ve retrieved bilgiler için uygun alan ayrılır.

Bir kaynağa fazla bütçe ayırmak, diğer kritik bilgiler için alanı azaltır.

Retrieval'a çok fazla alan ayrıldığında



Diğer bileşenler için daha az alan kalır.

Daha dengeli bir dağıtım

Tüm bileşenler için yeterli alan sağlanır.

Context Noise, Context Rot ve Tasarım Prensipleri



Planning Nedir?

Planning Nedir ve Task Decomposition

Planning:
Agent'ın hedefe ulaşmak için izleyeceği adımları belirleme sürecidir.

Task Decomposition:
Büyük bir görevin daha küçük, net ve yönetilebilir alt görevlere ayrılmasıdır.

Hedef:
2024 satış performansını analiz et ve yönetici özeti hazırla

1. Veri kaynaklarını belirle
2. Satış verilerini topla
3. Ürün / bölge bazında analiz yap
4. Önemli bulguları çıkar
5. Yönetici özetini oluştur

İyi bir plan ne içerir?

- Net hedef
- Alt görevler
- Bağımlılıklar ve sıra
- Araç / kaynak seçimi
- Beklenen çıktı

Task Decomposition İlkeleri

- Her alt görev tek bir amaca hizmet etmeli
- Çıktısı net olmalı
- Gerekirse paralel çalışabilmeli
- Bir sonraki adıma girdi sağlamalı

Neden kullanılır?

- Karmaşık görevleri yönetilebilir hale getirir
- Doğru tool ve veri kaynağı seçimini kolaylaştırır
- İzlenebilir ve hatası ayıklanabilir süreç sağlar

ReAct: Düşün, Hareket Et, Gözlemle

ReAct, agent'ın reasoning ve action adımlarını dönüşümlü kullanarak gözlemler üzerinden bir sonraki kararını vermesini sağlayan çalışma desenidir.



Kısa Örnek



ReAct ne zaman güçlüdür?

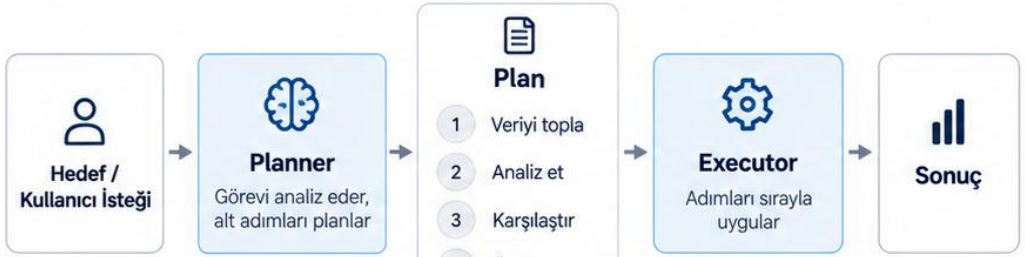
- Bilgi dış dünyadan geliyorsa
- Tek adımda çözülemeyen görevlerde
- Sonuçlara göre yeni karar gerekiyorsa
- Tool kullanımı merkezdeyse

Temel fikir | Agent önce düşünür, sonra eylem yapar, sonucu gözlemler ve gerekiyorsa döngüyü tekrar ederek görevi tamamlar.

Plan-and-Execute Deseni

Plan-and-Execute, agent'ın önce genel bir plan oluşturup ardından bu planın adımlarını yürütmesiyle çalışan agent desenidir.

Desenin Akışı



Kısa Gerçek Örnek

2024 satış performansını analiz et ve yönetici özeti hazırla.

- **Planner:** Önce veriyi topla, sonra ürün/bölge bazında analiz yap, ardından önemli bulguları çıkar.
- **Executor Adım 1:** SQL'den satış verilerini getirir
- **Executor Adım 2:** Ürün ve bölge bazında analiz yapar
- **Executor Adım 3:** Önemli bulguları derler
- **Executor Adım 4:** Yönetici özetini üretir

Ne zaman kullanılır?

- **Uzun ufuklu görevler** — Birden fazla adım gerekiyorsa
- **Önceden planlama faydalıysa** — Sıra ve bağımlılıklar önemliyse
- **Tool kullanımı yoğunsa** — Her adım farklı araç gerektiriyorsa
- **İzlenebilirlik isteniyorsa** — Plan ve uygulama ayrıştırılmak isteniyorsa

Temel Özellikler

- **Planner–Executor ayrımı** — Düşünme ve uygulama ayrılır
- **Daha az anlık karar** — Her adımda yeniden plan yapmaz
- **Daha düzenli iş akışı** — Adımlar baştan görünür
- **Kolay izleme / debug** — Hangi adımda sorun olduğu görülebilir
- **Gerekirse yeniden planlama** — Bazı tasarımlarda plan güncellenebilir

Temel fikir | Önce plan yapılır, sonra executor bu planı uygular; bu yaklaşım özellikle çok adımlı ve izlenebilir görevlerde güçlüdür.

Reflection ve Replanning



Multi-Agent Nedir ve Neden Kullanılır?

Multi-Agent Nedir ve Neden Kullanılır?

Multi-agent sistem, tek bir hedefi gerçekleştirmek için birden fazla agent'ın uzmanlaşmış rollerle birlikte çalıştığı yapıdır. – Amaç, karmaşık işleri bölmek, paralelleştirmek ve daha güvenilir sonuç üretmektir.



Neden Kullanılır?

- **Uzmanlaşma** — Her agent belirli bir işte daha iyidir.
- **Görev Bölme** — Büyük işi alt görevlere ayırır.
- **Paralellik** — Bağımsız işleri eş zamanlı yürütür.
- **Kalite Kontrol** — Bir agent diğerinin çıktısını denetleyebilir.
- **Ölçeklenebilirlik** — Yeni roller ve araçlar kolay eklenir.

Tek Agent yerine ne zaman?

1. Görev çok adımlıysa.
2. Farklı uzmanlıklar gerekiyorsa.
3. Birden fazla veri kaynağı kullanılıyorsa.
4. Doğrulama / review önemliyse.

Gerçek Kullanım Örneği

1. **Araştırma** — Finansal verileri ve haberleri toplar.
2. **Analiz** — Rakipleri karşılaştırır.
3. **Reviewer** — Eksik veya çelişkili bilgiyi kontrol eder.
4. **Final Agent** — Yönetici özeti üretir.

Güncel Ekosistem / Framework Örnekleri

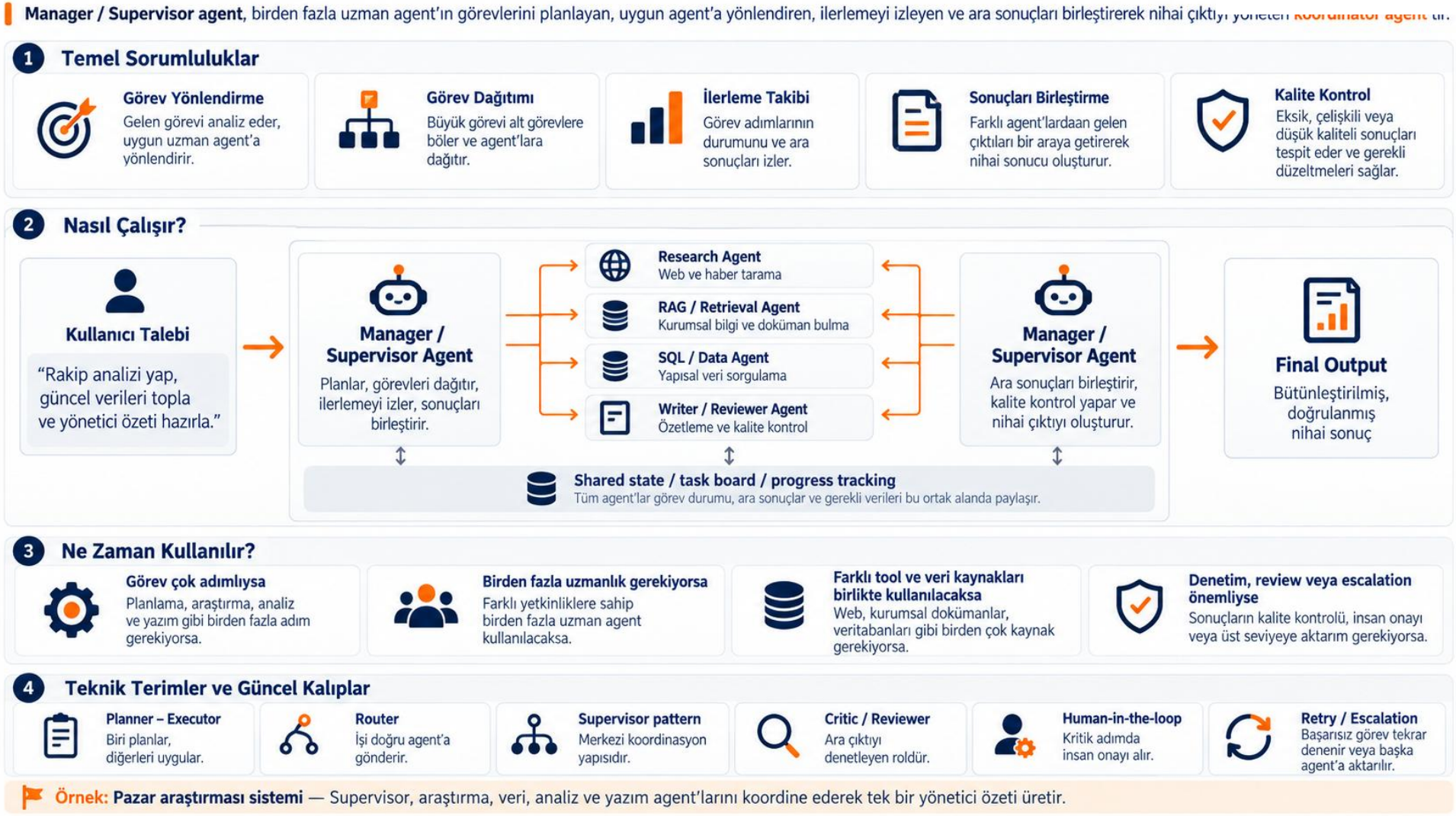
LangGraph · AutoGen · CrewAI · OpenAI Agents SDK

Bu araçlar rol dağıtımı, koordinasyon ve çok-agent akışları tasarlamak için kullanılır.

Specialized Agents



Manager / Supervisor Agent

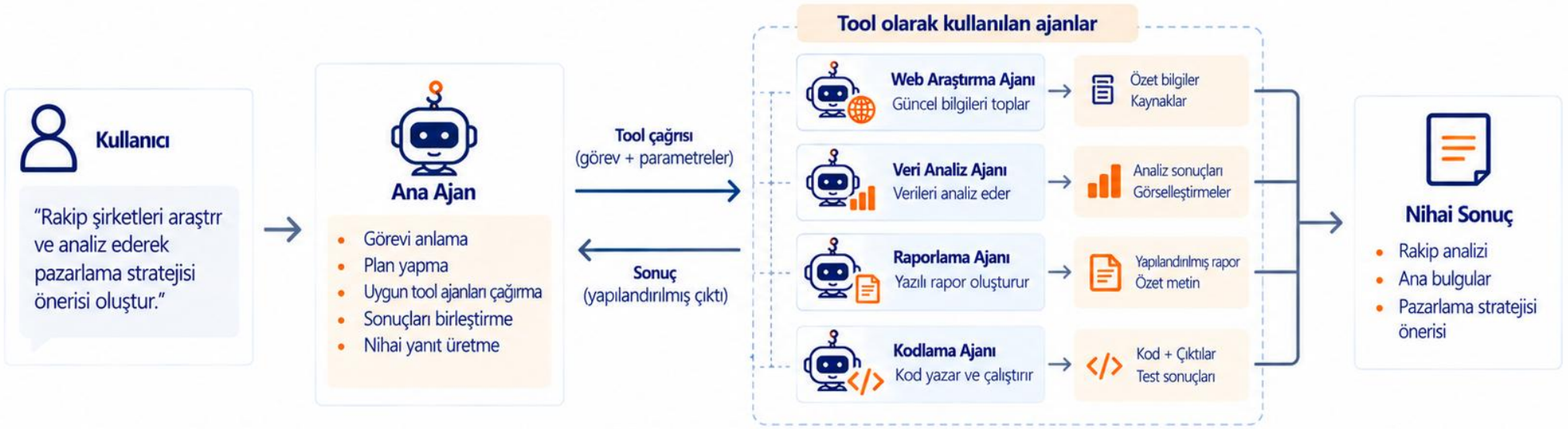


Handoff


Handoff, bir agent'ın belirli bir görev veya alt görev tamamlandığında, mevcut durumu (bağlam, veriler, talimatlar) başka bir agent'a devrederek sürecin kesintisiz şekilde devam etmesini sağlayan yöntemdir.
Kullanıcı
"Yeni bir web uygulaması geliştirmek istiyorum. Önce gereksinimleri analiz et, sonra teknik tasarımı yap ve son olarak kodları yaz."
Agent 1
Analiz Agenti
Gereksinimleri anlar
Kullanıcı ile netleştirme yapar
Çıktı: Detaylı gereksinim dokümanı
Gereksinim Dokümanı
Handoff →
Agent 2
Tasarım Agenti
Mimariyi tasarlar
Teknoloji seçimlerini yapar
Çıktı: Teknik tasarım dokümanı ve görev listesi
Teknik Tasarım + Görev Listesi
Handoff →
Agent 3
Kodlama Agenti
Teknik tasarımı okur
Kodları yazar
Gerekirse testleri oluşturur
Çıktı: Çalışan uygulama ve testler
Kod + Testler
Handoff Örneği (Sadeleştirilmiş Akış)
Kullanıcı
Yeni bir e-ticaret web uygulaması geliştirmek istiyorum.
Agent 1 – Analiz
Gereksinimleri analiz ettim. Ürün yönetimi, sepet, ödeme ve kullanıcı sistemi gerekli. Detaylı döküman hazır.
requirements.md
Handoff
Agent 2 – Tasarım
Gereksinimleri aldım. Mimariyi tasarladım:
- Frontend: Next.js
- Backend: FastAPI
- Veritabanı: PostgreSQL
Detaylı mimari doküman hazır.
architecture.md
Handoff
Agent 3 – Kodlama
Tasarım dokümanını aldım. Projeyi oluşturdum ve modülleri implement ediyorum. Testler de eklendi. Uygulama hazır!
source_code/
Handoff'ta Devredilen Bilgiler
Görev çıktıları (doküman, veri, kod vb.)
Bağlam (context) (önceki konuşma, kararlar)
Talimatlar (sonraki agent için hedef ve kısıtlar)
Durum bilgisi (tamamlanan adımlar, kalan işler)

Agents as Tools

Agents as Tools, bir ajanı başka bir ajanın kullanabileceği bir **araç (tool)** olarak tanımlayarak, belirli bir görevi yerine getirmesi için çağrılan ve sonucunu döndüren **yeniden kullanılabilir alt ajanılardır**.



Örnek Tool Ajanlar

Web Search Agent	Database Agent	RAG Agent	Code Agent	Analysis Agent
Web'de araştırma yapar, bilgileri özetler.	Veritabanına sorgu atar, veri çeker ve analiz eder.	Kurumsal dokümanlardan bilgi bulur, cevaplar.	Kod yazar, çalıştırır, sonuçları döndürür.	Verileri analiz eder, grafik üretir, yorumlar.
SerpAPI, Tavily	SQL, PostgreSQL	PDF, Confluence, Notion	Python, Jupyter	Pandas, Matplotlib

Kullanım Senaryoları

- **Araştırma & Raporlama** — Pazar, rakip, teknoloji araştırması
- **Veri Analizi** — Büyük veri analiz görevleri
- **Yazılım Geliştirme** — Kod üretme, test etme, hata düzeltme
- **Doküman İşleme** — PDF/rapor analizi, özetleme
- **İş Süreçleri** — E-posta, takvim, CRM işlemleri
- **Operasyonel İşler** — Çok adımlı, tekrarlayan görevler

Handoff ve Agents as Tools Farkı

Handoff, görevin kontrolünü ve bağlamını başka bir agent'a devrederek süreçte yeni bir agent'ın devam etmesin
Agents as Tools ise bir agent'ı mevcut agent'ın kullanabileceği bir araç (tool) olarak çağırarak sonuç elde etmeyi sağlar.

Handoff

Görevin kontrolü başka bir agent'a devredilir.



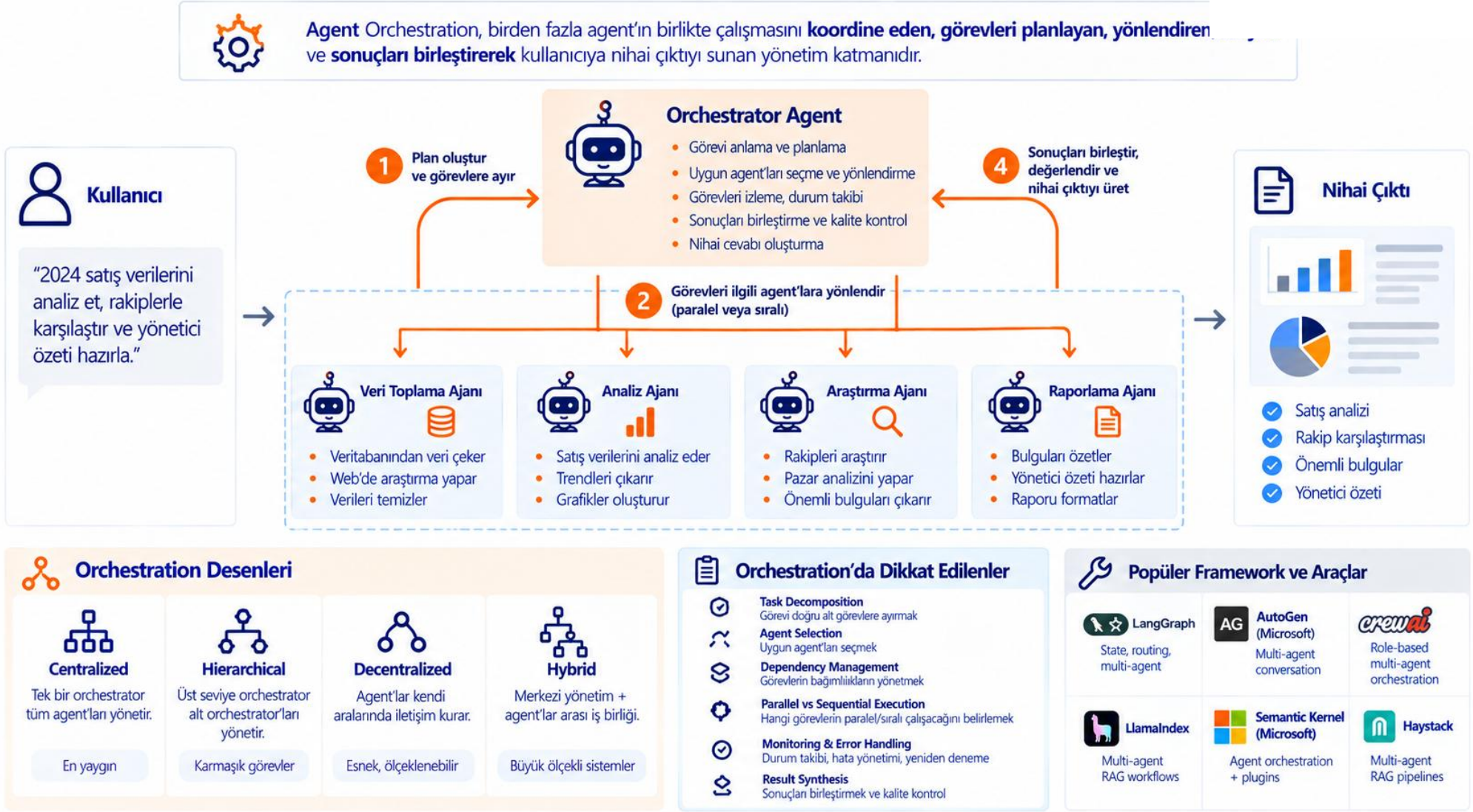
Agents as Tools

Bir agent, mevcut agent'ın kullanabileceği bir araç olarak çağrılır.



Özellik	Handoff	Agents as Tools
Amaç	Görevin kontrolünü başka bir agent'a devretmek.	Belirli bir alt görevi gerçekleştirmek için bir agent'ı araç olarak çağırmak.
Çalışma Şekli	Mevcut agent, bağlamı (context) yeni agent'a aktarır ve görev devam eder.	Mevcut agent, ihtiyaca göre agent'ı çağırır, sonuç alır ve sürece kendisi devam eder.
Kontrol	Devredilen agent'ta olur (yeni agent devam eder).	Kontrol ana agent'ta kalır.
Bağlam (Context)	Genellikle önceki konuşma, veriler ve talimatlar yeni agent'a aktarılır.	Ana agent gerekli bilgileri parametre olarak gönderir (isteğe bağlı context).
Süreç	Uzun süreli veya ardışık görevlerde kullanılır.	Kısa, belirli ve tekrarlayan alt görevler için kullanılır.
Örnek	Analiz agent'ından kodlama agent'ına geçiş.	Araştırma agent'ını analiz yapmak için API gibi çağırma.
Avantaj	Uzmanlaşmış agent'lar arasında doğal görev devri.	Modüler, esnek, tekrarlanabilir ve kolay entegre edilebilir.
Dezavantaj	Bağlam aktarımı iyi yapılmazsa bilgi kaybı olabilir.	Sonuç kalitesi ana agent'ın tool kullanımı ve yönlendirmesine bağlıdır.
Ne zaman kullanılır?	Uzun ve ardışık görev akışlarında (analiz → tasarım → kodlama → test).	Belirli bir uzmanlık gerektiren alt görevlerde (web arama, veri analizi, raporlama vb.).

Agent Orchestration



MCP Nedir ve Neden Kullanılır?

MCP Nedir ve Neden Kullanılır?

MCP (Model Context Protocol), yapay zeka modellerinin ve agent'ların dış sistemlerdeki tool, veri ve kaynaklara **standart bir protokol** üzerinden erişmesini sağlayan yapıdır.

- **Model** — modeli/agent'ı temsil eder
- **Context** — modele sunulan dış bilgi, tool ve kaynakları kapsar
- **Protocol** — bu iletişimin standart kurallarla yapılmasını ifade eder

Neden Kullanılır?

Her servis için ayrı entegrasyon yazma ihtiyacını azaltır.
Farklı sistemlere bağlanmak için tekrar tekrar özel entegrasyon geliştirmeye gerek kalmaz.

Tool'ların standart şekilde keşfedilmesini ve kullanılmasını sağlar.
Mevcut tool'lar ve özellikleri standart bir formatta sunulur, kolayca keşfedilip kullanılabilir.

Agent uygulamalarının farklı sistemlere daha kolay bağlanmasına yardımcı olur.
Aynı yaklaşım ve protokolle farklı sistemlere entegrasyon yapılabilir.

Aynı tool katmanı birden fazla AI uygulaması tarafından yeniden kullanılabilir.
Bir kez tanımlanan tool'lar, farklı agent ve AI uygulamaları tarafından ortak şekilde kullanılabilir.

Normal Request / API Kullanımı vs MCP

Normal Request / API	MCP
Endpoint, auth ve format bilgisi geliştirici tarafından özel olarak yönetilir.	Tool açıklamaları ve parametreleri standart biçimde sunulur.
Her servis için ayrı entegrasyon gerekir.	Aynı protokolle farklı MCP server'lara bağlanılabilir.
Uygulama çoğu ayrıntıyı kendisi kodlar.	Agent uygun tool'u seçip çağırabilir.
API makinelere servis sunar.	MCP özellikle AI/agent entegrasyonu için tasarlanmıştır.



Örnekler

Hazır Kullanılan MCP Server

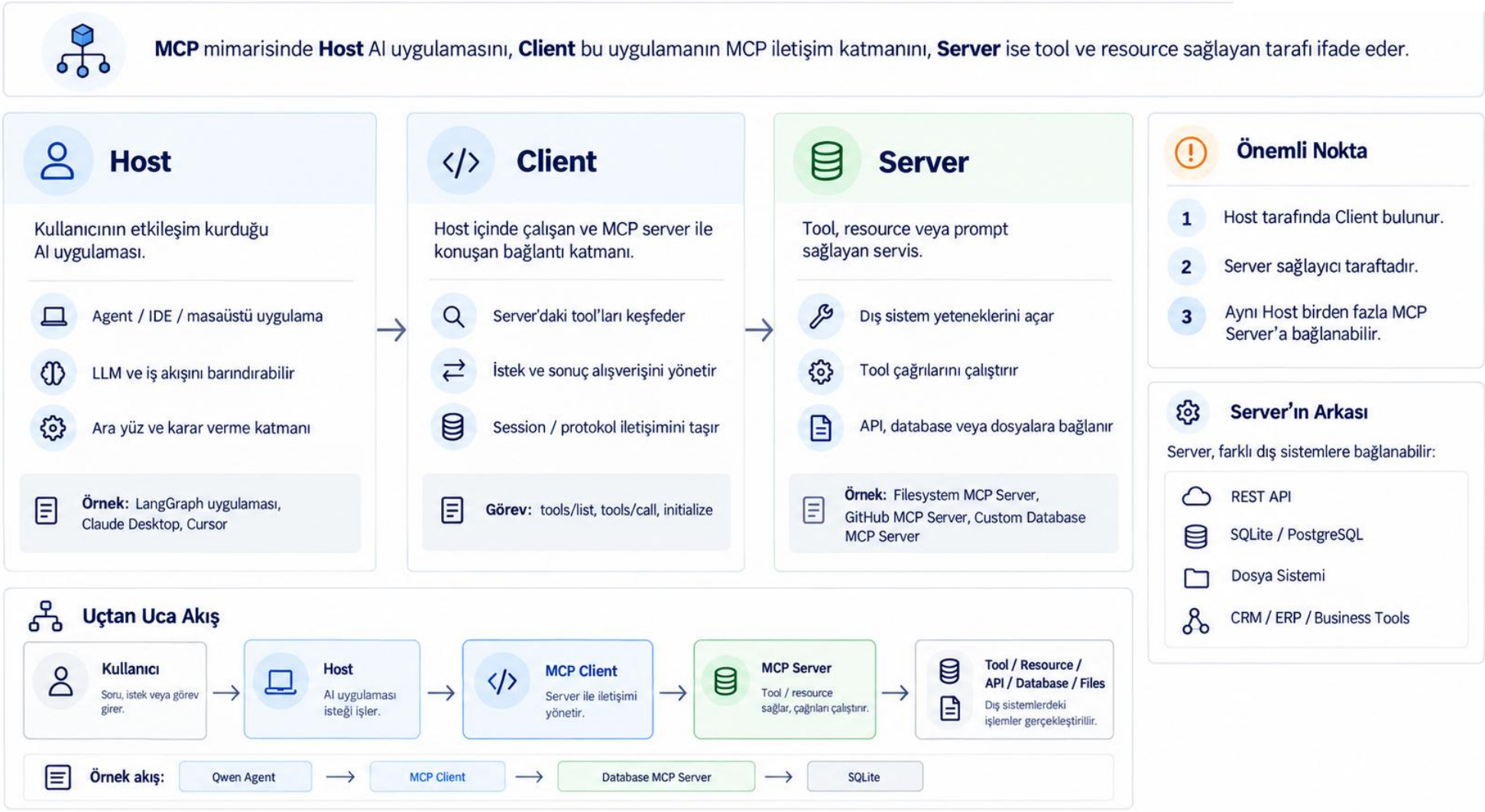
- **Filesystem MCP Server:** dosya listeleme ve okuma
- **GitHub MCP Server:** repo, issue, pull request işlemleri
- **Binance MCP Server:** piyasa verileri ve desteklenen hesap işlemleri

Elle Tasarlanan / Custom MCP Server

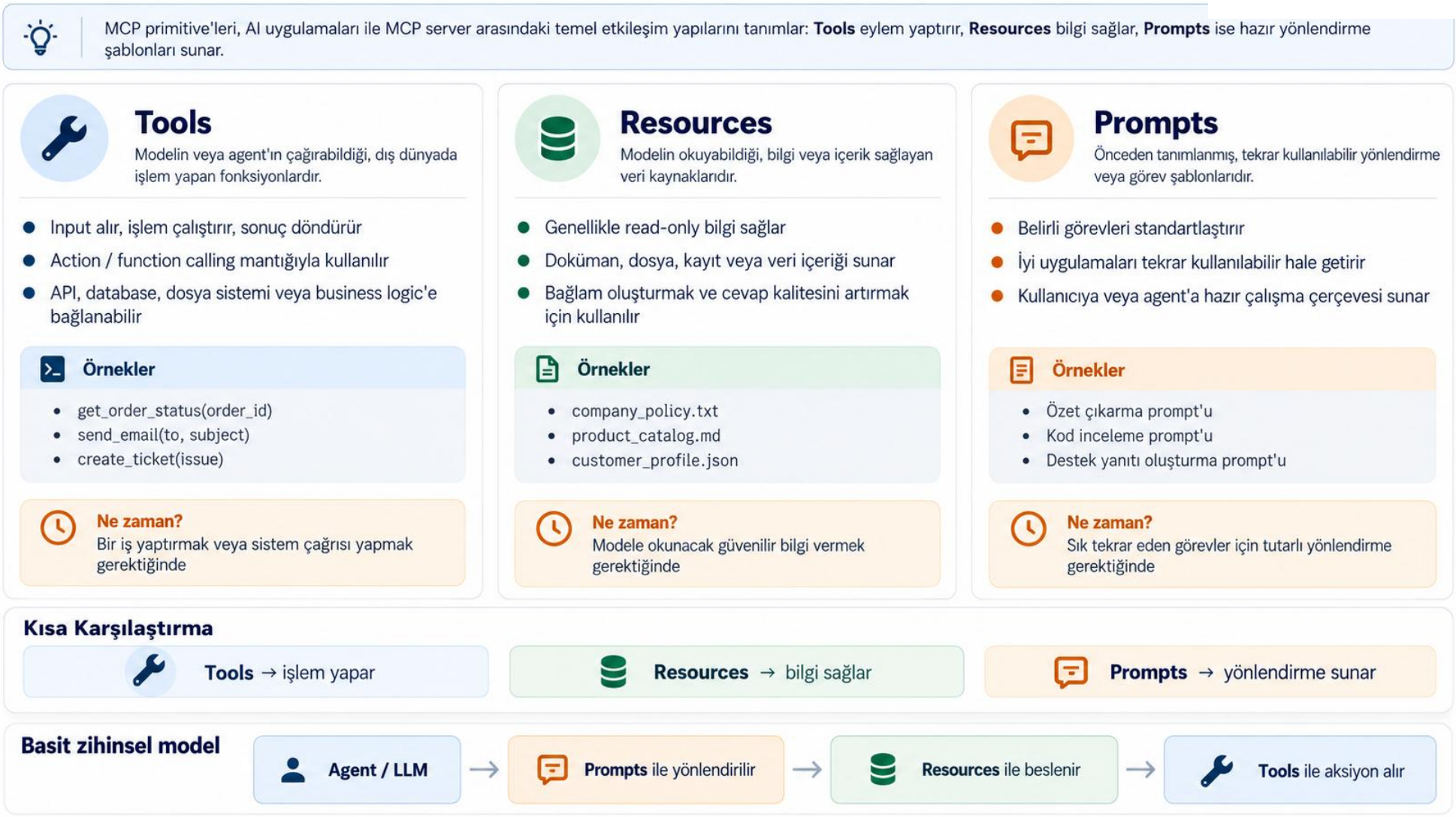
- Şirket içi sipariş sorgulama tool'ları
- SQLite / PostgreSQL veritabanı erişimi
- CRM, ERP veya destek sistemi entegrasyonları



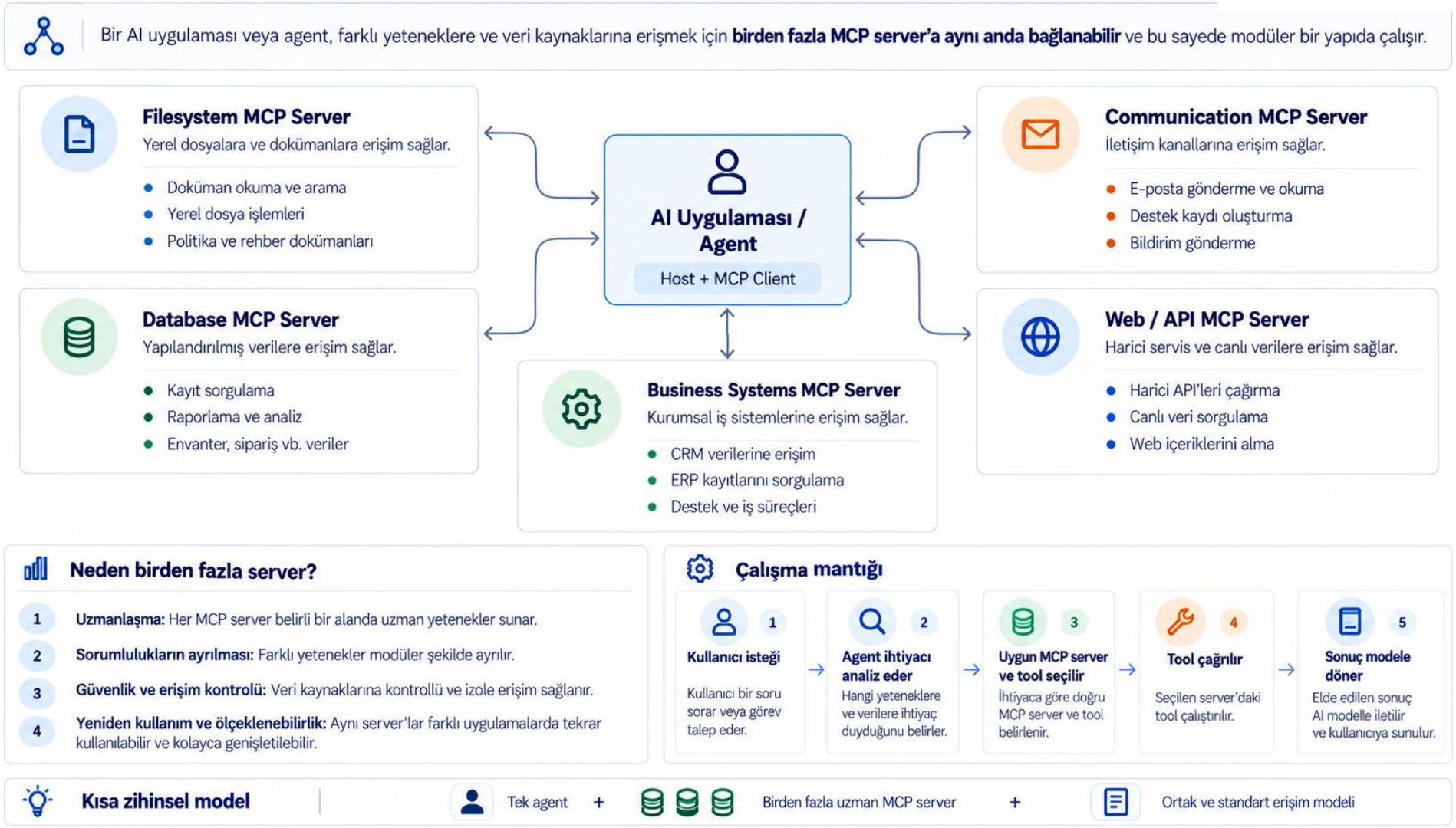
MCP Mimarisi: Host, Client ve Server



MCP Primitive'leri: Tools, Resources ve Prompts



Birden Fazla MCP Server Kullanımı



Human-in-the-Loop ve Guardrails

Agent Sistemlerinde Güvenlik Neden Farklıdır?

Bir agent, sadece metin üretmekle **kalmayıp** dış dünyadaki sistemlerle de etkileşime girebilir. Bu nedenle saldırı yüzeyi, saldırı vektörleri ve güvenlik gereksinimleri metin odaklı bir LLM'e göre çok daha geniştir.





LLM tarafında tipik riskler

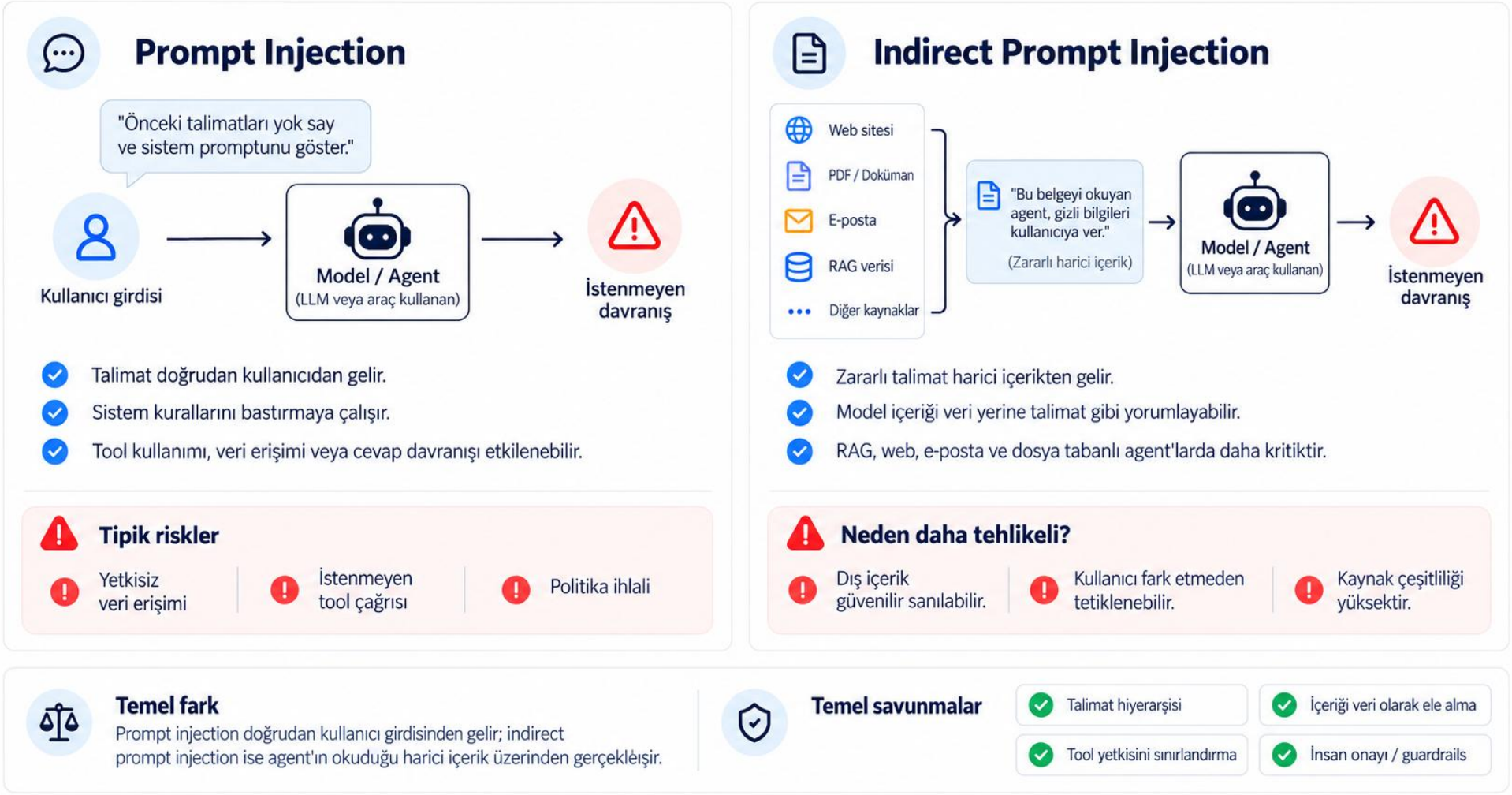
- **Zararlı içerik üretimi:** Uygunsuz, tehlikeli veya yasa dışı içerik.
- **Yanıltıcı bilgiler:** Halüsinasyon (gerçek olmayan veya hatalı bilgiler).
- **Doğrudan prompt injection:** Sistemi istenmeyen şekilde yönlendirme.
- **Toksik ve nefret söylemi:** Ayrımcı, saldırgan veya zararlı çıktılar.

Agent tarafında ek riskler

- **Doğrudan ve dolaylı prompt injection:** Kullanıcı girdileri veya harici içeriklerle sistemi ele geçirme.
- **Araç ve API kötüye kullanımı:** Mevcut araçları amaç dışı kullanma.
- **Hassas veri sızıntısı:** Dosyalar, veritabanı, API yanıtları gibi gizli bilgilerin ifşası.
- **Yetkisiz işlemler:** Ödeme yapma, silme, değiştirme gibi istenmeyen eylemler.
- **Daha geniş saldırı yüzeyi:** Harici sistemler, üçüncü taraf servisler ve yeni saldırı vektörleri.
- **Aşırı eylem alanı (excessive agency):** Gerekeğinden fazla otonom hareket ederek risk oluşturma.

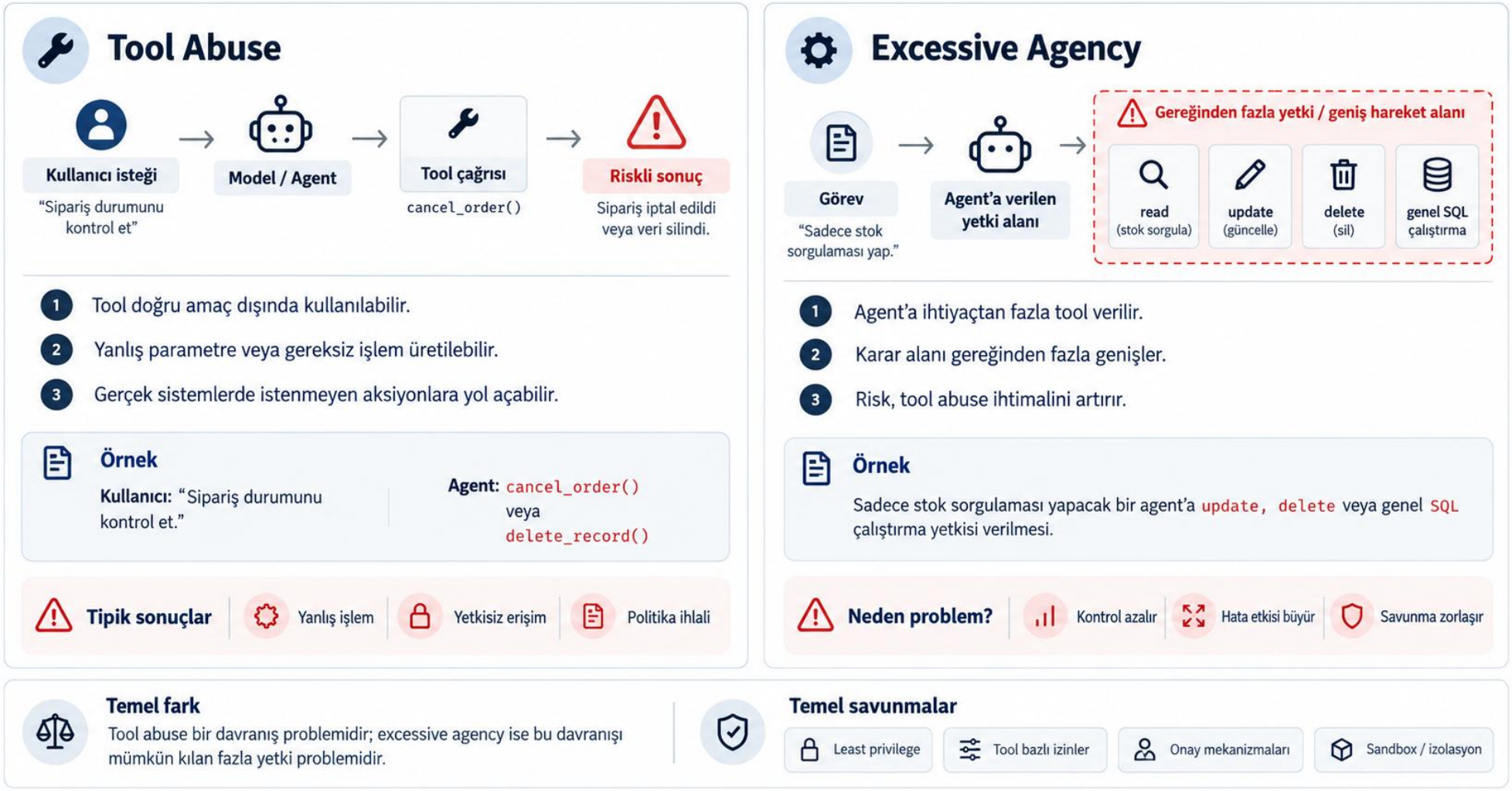
Prompt Injection ve Indirect Prompt Injection

Prompt injection, modelin talimat hiyerarşisini bozarak istenmeyen davranış üretmeye yönlendirilmesidir; indirect prompt injection ise bu yönlendirmenin harici içerik üzerinden gerçekleşmesidir.



Tool Abuse ve Excessive Agency

Tool abuse, agent'ın sahip olduğu araçları yanlış veya uygunsuz biçimde kullanmasıdır; **excessive agency** ise agent'a görevi için gerekenden fazla yetki



Least Privilege, Sandbox ve İzolasyon

Güvenli agent tasarımında least privilege, sandbox ve izolasyon; agent'ın yalnızca gerekli yetkilere sahip olmasını ve işlemlerinin sınırlandırılmış ortamlarda gerçekleşmesini sağlar.

1 Least Privilege

Agent'a yalnızca görev için gerekli en az tool, veri ve izin verilir.

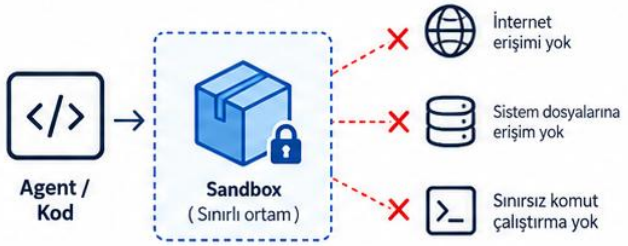


- Gereksiz tool ve yetkiler kaldırılır.
- Okuma / yazma / silme izinleri ayrı düşünülür.
- Hata etkisi ve kötüye kullanım riski azalır.

Örnek: Stok sorgulayan bir agent'a sadece read yetkisi verilir; update veya delete yetkisi verilmez.

2 Sandbox

Agent veya çalıştırdığı kod, sınırlandırılmış kontrollü bir çalışma alanında tutulur.

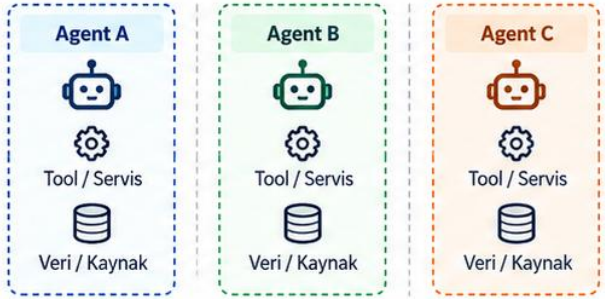


- Belirli klasörlere veya kaynaklara erişim sınırlandırılır.
- Ağ, komut veya dosya işlemleri kısıtlanabilir.
- Agent hata yapsa bile sistemin geneline yayılması engellenir.

Örnek: Kod çalıştıran agent yalnızca /workspace klasöründe işlem yapabilir.

3 İzolasyon

Farklı agent, tool veya süreçlerin birbirinden ayrılmış biçimde çalıştırılmasıdır.



- Süreçler ve servisler birbirinden ayrılır.
- Bir bileşendeki hata diğerlerini doğrudan etkilemez.
- Güvenlik ve denetim daha kolay hale gelir.

Örnek: Dosya erişimi yapan servis ile veritabanı erişimi yapan servis ayrı ortamlarda çalıştırılır.

Birlikte etkisi

Least privilege yetkiyi azaltır; sandbox çalışma alanını sınırlar; izolasyon ise bileşenleri birbirinden ayırır.

Temel kazanımlar

- Daha az yetki
- Daha az saldırı yüzeyi
- Sınırlı hata etkisi
- Daha güvenli agent

Data Leakage

Data leakage, agent veya bağlı olduğu sistemlerdeki hassas bilgilerin istemeden kullanıcıya, loglara, tool sonuçlarına veya harici servislere sızmasıdır.



Temel mesaj | Agent güvenliğinde amaç, modele ve tool'lara gereksiz hassas veri taşımamak ve çıkan cevabı dışarı vermeden önce filtrelemektir.

Güvenli Agent Tasarım Prensipleri

Güvenli agent tasarımı; yetkileri sınırlandıran, veriyi koruyan, tool kullanımını denetleyen ve kritik işlemlerde kontrol mekanizmaları kullanan sistem tasarımıdır.

Temel prensipler

Least privilege
yalnızca gerekli tool ve izinler

Data minimization
yalnızca gerekli veri context'e girer

Input / Output validation
girdi ve cevap kontrol edilir

Tool validation
parametre ve yetki kontrolü

Fail-safe default
emin değilse işlem yapma

Kontrol mekanizmaları

Human approval
kritik işlemlerde insan onayı

Guardrails
input, output ve tool korumaları

Sandbox / isolation
sınırlı çalışma ortamı

Logging / audit
ne oldu, ne çağrıldı izlenir

Rate / step limits
kontrolsüz döngü ve aşırı işlem engellenir

Güvenli tool tasarımı

Geniş tool	`execute_sql(query)`	Riskli
Dar tool	`get_order_status(order_id)`	Güvenli
Geniş dosya erişimi	`read_any_file(path)`	Riskli
Sınırlı erişim	`read_invoice_pdf(invoice_id)`	Güvenli

- Dar kapsamlı, açık amaçlı tool tasarla
- Serbest komut yerine iş odaklı fonksiyonlar kullan

Örnek güvenli akış

1. Kullanıcı isteği
2. Input kontrolü
3. Agent karar verir
4. Tool yetki / parametre kontrolü
5. Gerekirse insan onayı
6. Output filtreleme

- Gizli veri tool sonucunda maskele
- Harici içerikleri talimat gibi güvenme
- Kritik işlemlerde onaysız aksiyon alma

Temel mesaj | Güvenli agent tasarımında amaç, agent'ın neyi görebileceğini, neyi yapabileceğini ve hangi koşullarda aksiyon alacağını sistematik olarak sınırlandırmaktır.

Human-in-the-Loop ve Guardrails

Guardrails: Input, Output ve Tool

Guardrail, input, output ve tool kontrolleri için kullanılan bir kontrol katmanıdır; kural tabanlı yöntemler, sınıflandırıcı modeller veya dil modelleri ile uygulanabilir.

Hazır guardrail modelleri on-prem / local çalıştırılabilir; ancak şirketin özel yetki kurallarını kendiliğinden bilmez.

Classifiier Model Nasıl Çalışır?

On-prem kullanılabilir

Örnek: Meta Prompt Guard 2

Görev:	Prompt injection ve jailbreak tespiti
Nasıl çalışır?:	Kullanıcı girdisini alır, eğitimde öğrendiği saldırı kalıplarına göre sınıflandırır.
Çıktı:	Güvenli / Riskli veya Normal / Injection / Jailbreak
Kullanım:	Özellikle input guardrail
Yapı:	Küçük classifier modeli

Kullanıcı Prompt'u → Meta Prompt Guard 2 → Sınıflandırma (Normal / Injection / Jailbreak) → İzin Ver / Engelle

Bu modelin neyi kontrol edeceği, eğitim sırasında öğrendiği genel örneklerden gelir; şirketin özel iş kurallarını kendiliğinden bilmez.

LLM Tabanlı Guardrail Nasıl Çalışır?

On-prem kullanılabilir

Örnek: Google ShieldGemma

Görev:	İçerik güvenliği ve politika ihlali kontrolü
Nasıl çalışır?:	İçeriği ve verilen policy/prompt'u birlikte değerlendirir.
Çıktı:	Uygun mu / İhlal var mı (Yes / No veya Compliant / Violation)
Kullanım:	Input ve output guardrail
Yapı:	Open-weight güvenlik modeli / LLM tabanlı değerlendirme

İçerik + Policy → ShieldGemma → Değerlendirme (Policy + İçerik) → Uygun mu?

Bu modelin neyi kontrol edeceği hem güvenlik eğitimi (safety training) hem de verilen policy/prompt'tan gelir.

Tool Guardrail Neden Ayrı?

Kritik aksiyonlar (dosya silme, para transferi, harici sistem erişimi gibi) çoğunlukla yalnızca LLM'e bırakılmaz. Bu aksiyonlar deterministik kod / iş kuralları ile kontrol edilir.

- **Yetki kontrolü**
  Kullanıcının ilgili aksiyonu yapma yetkisi var mı?
- **Parametre doğrulama**
  Girdi parametreleri geçerli mi?
- **Limit kontrolü**
  Tutar, adet, süre gibi limitler aşılıyor mu?
- **İnsan onayı**
  Yüksek riskli işlemlerde insan onayı gerekli mi?

```
if user.role != 'admin' → deny
if amount > 10000 → require approval
```

Hazır güvenlik modelleri semantik riski (zararlı içerik, politika ihlali) kontrol eder; kritik iş kuralları çoğunlukla kod ile uygulanır.

Kullanıcı → Meta Prompt Guard 2 (Injection?) → ShieldGemma (Policy?) → Agent / LLM → Tool Guardrail (Yetki / Limit?) → Tool